

**SERVICIO NACIONAL DE APRENDIZAJE - SENA
CENTRO TECNOLOGÍA Y MANUFACTURA AVANZADA**

**GUÍA DE TRABAJO: DOCKER + GIT
4 ESCENARIOS PROGRESIVOS**

**Aprendiz: Manuela Córdoba Robledo
Ficha: 3229209**

**Programa: Análisis y Desarrollo de Software (ADSO)
Competencia: Contenerización de Aplicaciones / DevOps
Instructor: Richard Alexander Betancur Sierra**

**Medellín, Colombia
21 de agosto de 2026**

Preparación del repositorio

Creé el repositorio `taller-docker-git` en GitHub y lo conecté con mi carpeta local usando `git init`, `git branch -M main` y `git remote add origin`. Verifiqué la conexión con `git remote -v`.

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git init
+ FullyQualifiedErrorId : UnexpectedToken

● PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git init
Initialized empty Git repository in C:/Users/roble/OneDrive/Desktop/taller-docker-git/.git/
● PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git branch -M main
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git remote add origin https://github.com/robledocordoba123-cmyk/taller-docker-git.git
● PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git remote -v
origin https://github.com/robledocordoba123-cmyk/taller-docker-git.git (fetch)
origin https://github.com/robledocordoba123-cmyk/taller-docker-git.git (push)
○ PS C:\Users\roble\OneDrive\Desktop\taller-docker-git>
```

Primer commit

Creé `.gitignore` (para excluir `.env` y evitar subir contraseñas al repositorio) y `README.md` con la descripción general del taller. Hice el primer commit siguiendo la convención `docs:` para cambios de documentación, y lo subí a GitHub con `git push -u origin main`.

```
● PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git add .
● PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   .gitignore
    new file:   README.md

PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git commit -m "docs: estru
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git commit -m "docs: estructura inicial del repositorio"
● PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 501 bytes | 167.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/robledocordoba123-cmyk/taller-docker-git.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

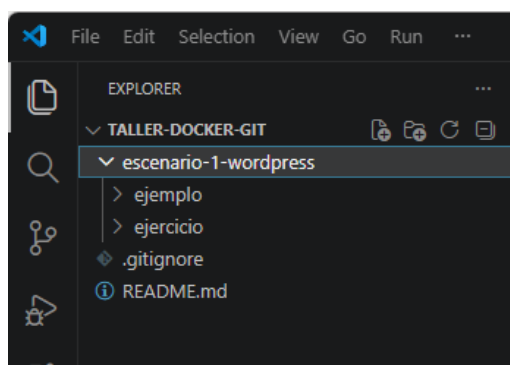
Escenario 1 — Creación de la rama

Creé la rama `escenario-1-wordpress` con `git checkout -b`, para desarrollar este escenario de forma aislada sin tocar `main`. Verifiqué con `git branch` que la rama activa (marcada con `*`) es la correcta.

```
branch 'main' set up to track 'origin/main'.
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git checkout -b escenario-1-wordpress
● Switched to a new branch 'escenario-1-wordpress'
● PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git branch
 * escenario-1-wordpress
   main
○ PS C:\Users\roble\OneDrive\Desktop\taller-docker-git>
```

Escenario 1 — Estructura de carpetas

Dentro de la rama `escenario-1-wordpress`, creé la carpeta del escenario con dos subcarpetas: `ejemplo/` (versión guiada de la guía, para estudiar) y `ejercicio/` (mi solución con los cambios pedidos).



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress> mkdir
r ejemplo
C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress

Mode                LastWriteTime         Length Name
----                -
d-----          20/08/2026   3:56 p. m.         ejemplo

PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress> mkdir
r ejercicio
```

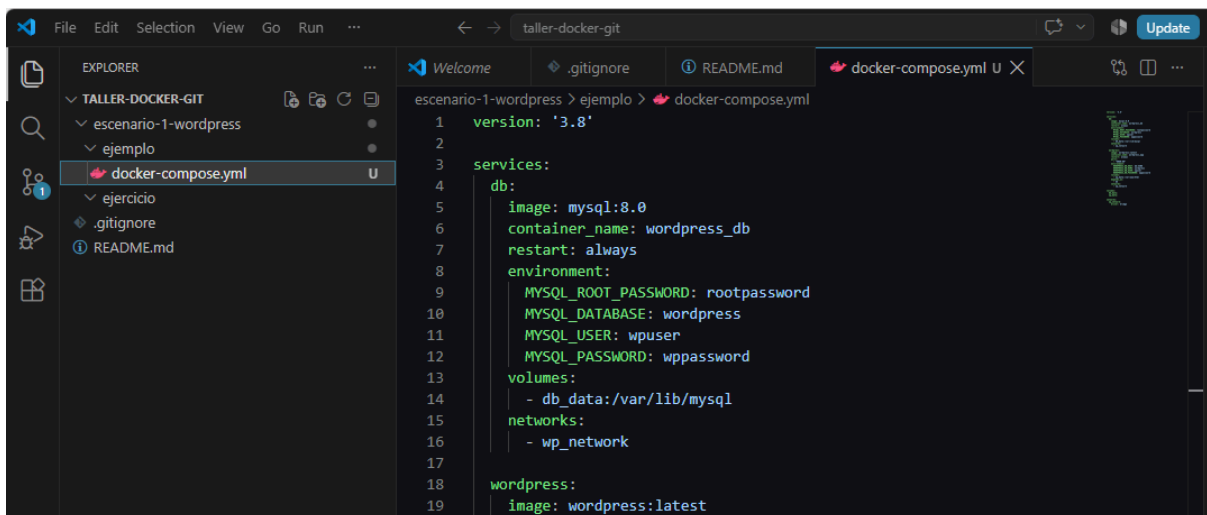
```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress> mkdir
r ejercicio

Directorio:
C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress

Mode                LastWriteTime         Length Name
----                -
d-----          20/08/2026   3:56 p. m.         ejercicio
```

Escenario 1 — Ejemplo guiado: docker-compose.yml

Creé el archivo `docker-compose.yml` dentro de `escenario-1-wordpress/ejemplo/`, siguiendo exactamente la guía: dos servicios (`db` con MySQL 8.0 y `wordpress`), conectados por la red `wp_network`, cada uno con su volumen (`db_data`, `wp_data`) para persistir los datos. El `depends_on`: `db` asegura que WordPress no arranque hasta que la base de datos esté lista.



```
1 version: '3.8'
2
3 services:
4   db:
5     image: mysql:8.0
6     container_name: wordpress_db
7     restart: always
8     environment:
9       MYSQL_ROOT_PASSWORD: rootpassword
10      MYSQL_DATABASE: wordpress
11      MYSQL_USER: wpuser
12      MYSQL_PASSWORD: wppassword
13     volumes:
14       - db_data:/var/lib/mysql
15     networks:
16       - wp_network
17
18   wordpress:
19     image: wordpress:latest
```

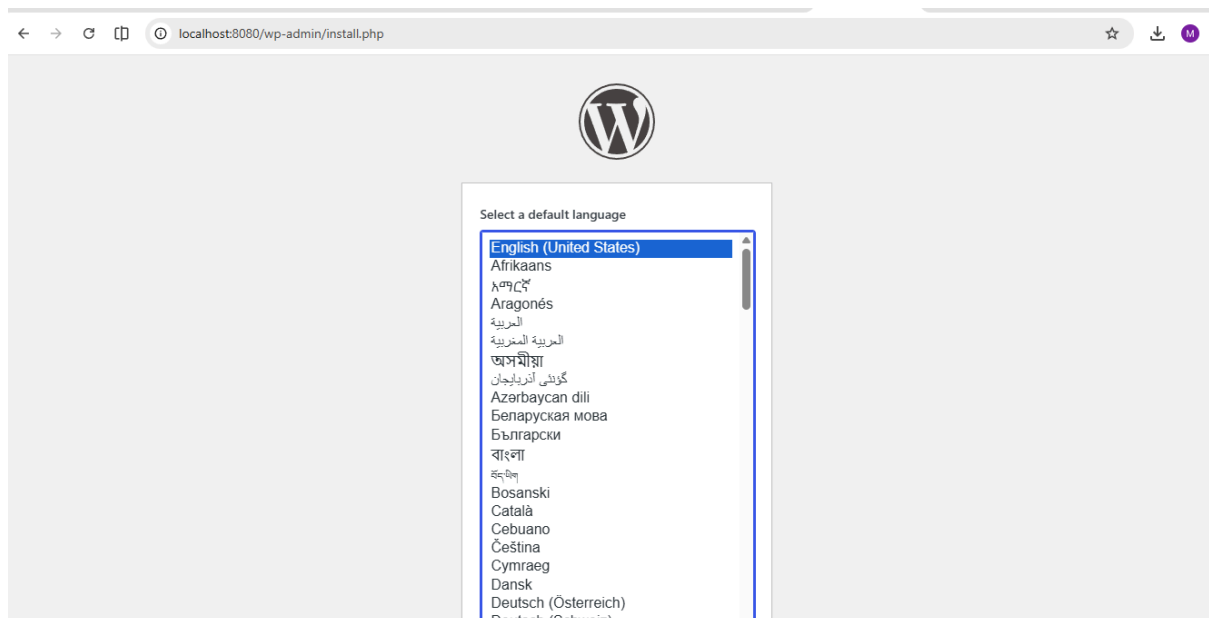
Escenario 1 — Obstáculo: error de conexión a la base de datos

Al levantar el ejemplo por primera vez, WordPress mostró "Error establishing a database connection". Esto ocurre porque `depends_on` solo espera a que el contenedor de la base de datos **arranque**, no a que MySQL termine de inicializarse internamente y esté listo para aceptar conexiones. Se resolvió esperando unos segundos y recargando la página. (Nota: en el Escenario 2 de la guía se usa un `healthcheck` para evitar este mismo problema de forma más robusta.)



Escenario 1 — Ejemplo funcionando

Tras esperar unos segundos a que MySQL terminara de inicializar, recargué `localhost:8080` y el instalador de WordPress cargó correctamente, confirmando que el contenedor `wordpress` logró conectarse al contenedor `db` a través de la red `wp_network`.



Escenario 1 — Commit y push del ejemplo

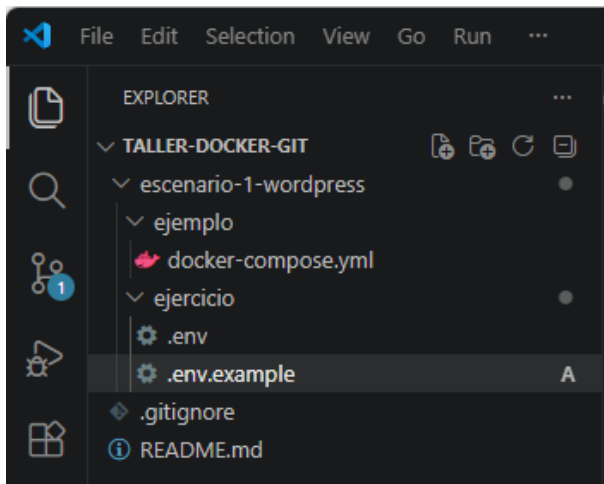
Detuve los contenedores con `docker-compose down`, volví a la raíz del repositorio, y subí el trabajo con `git add`, `git commit -m "feat: agrega ejemplo guiado de WordPress + MySQL"` y `git push origin escenario-1-wordpress`. La rama quedó publicada en GitHub, lista para el Pull Request final.

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress\ejemp
lo> docker-compose down
Desktop\taller-docker-git\escenario-1-wordpress\ejemplo\docker-compose.yml:
the attribute `version` is obsolete, it will be ignored, please remove it to avo
id potential confusion"
[+] down 3/3
✓ Container wordpress_app      Removed          16.2s
✓ Container wordpress_db      Removed          26.6s
✓ Network ejemplo_wp_network  Removed          1.5s
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress\ejemp
lo> cd ../../
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git add .
```

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git add .
>> git commit -m "feat: agrega ejemplo guiado de WordPress + MySQL"
>> git push origin escenario-1-wordpress
create mode 100644 escenario-1-wordpress/ejemplo/docker-compose.yml
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 792 bytes | 31.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'escenario-1-wordpress' on GitHub by visiting:
```

Escenario 1 — Verificación de seguridad del .env

Creé `.env` (con contraseñas reales) y `.env.example` (plantilla vacía) dentro de `ejercicio/`. Verifiqué con `git status` que solo `.env.example` aparece listo para commit — `.env` queda excluido gracias al `.gitignore`, confirmando que las credenciales nunca se subirán al repositorio.



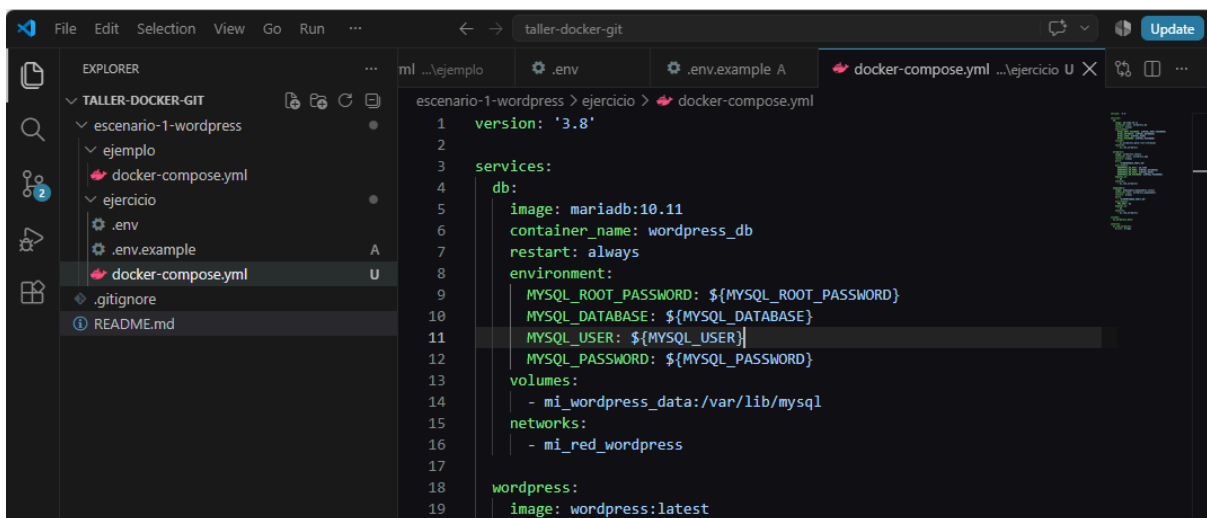
```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git status
escenario-1-wordpress/ejercicio/

nothing added to commit but untracked files present (use "git add" to track)
● PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git add .
● PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git status
On branch escenario-1-wordpress
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   escenario-1-wordpress/ejercicio/.env.example

○ PS C:\Users\roble\OneDrive\Desktop\taller-docker-git>
```

Escenario 1 — docker-compose.yml del ejercicio

Creé `docker-compose.yml` dentro de `ejercicio/` con los 4 cambios pedidos: MariaDB 10.11 en vez de MySQL, phpMyAdmin como tercer servicio, contraseñas tomadas del `.env` con `${VARIABLE}`, y nombres personalizados para el volumen (`mi_wordpress_data`) y la red (`mi_red_wordpress`).



Escenario 1 — Ejercicio levantado

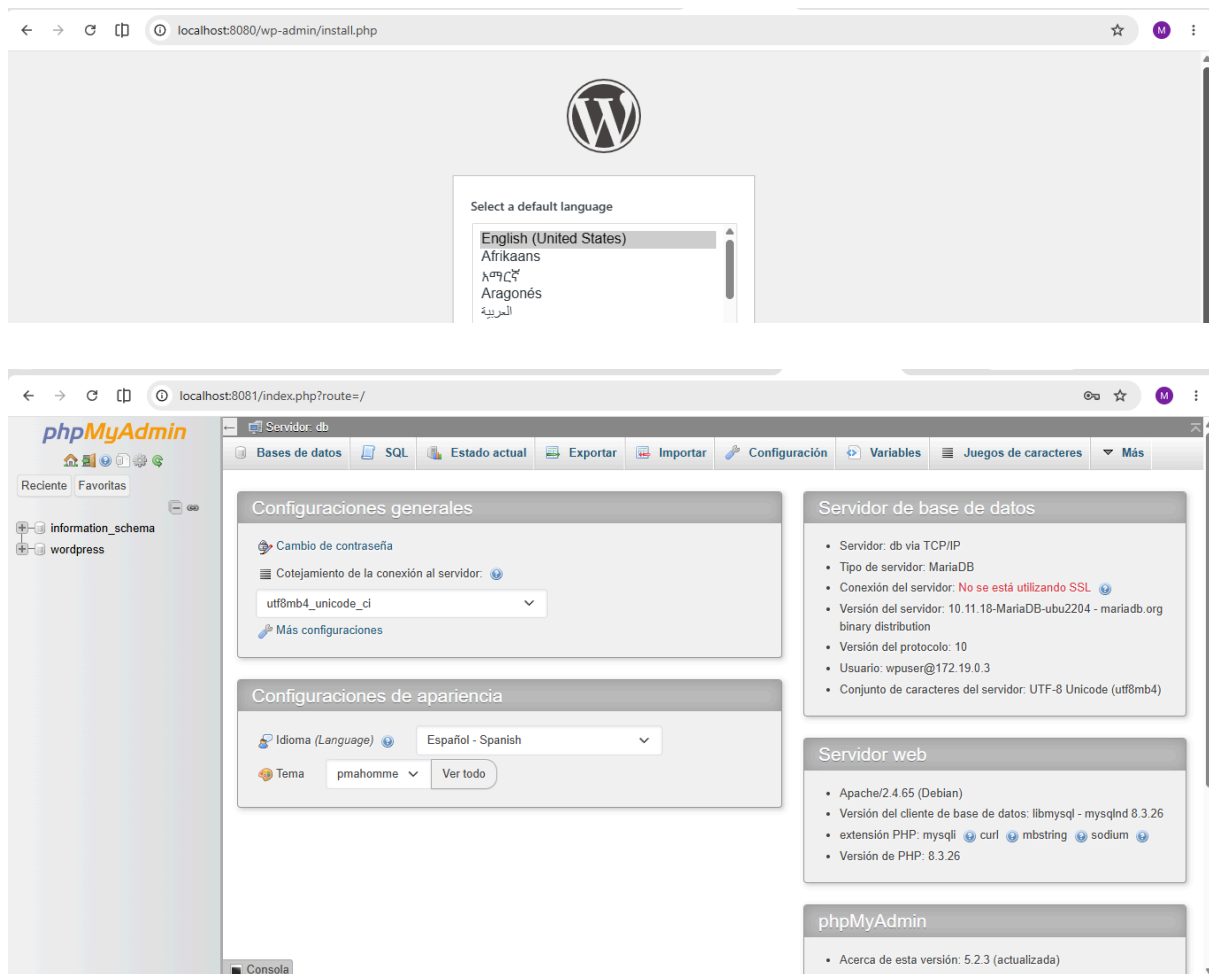
Con `docker-compose up -d` dentro de `ejercicio/`, se crearon la red `mi_red_wordpress`, el volumen `mi_wordpress_data`, y los 3 contenedores (`wordpress_db` con MariaDB, `wordpress_app`, `wordpress_phpmyadmin`). `docker-compose ps` confirma que los 3 están en estado `Up`.

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress\ejerc
icio> docker-compose up -d
time="2026-08-20T16:20:52-05:00" level=warning msg="C:\\Users\\roble\\OneDrive\\
Desktop\\taller-docker-git\\escenario-1-wordpress\\ejercicio\\docker-compose.yml
: the attribute `version` is obsolete, it will be ignored, please remove it to a
void potential confusion"
[+] up 38/38
✓ Image mariadb:10.11 Pulled 150.0s
✓ Image phpmyadmin/phpmyadmin:latest Pulled 233.1s
✓ Network ejercicio_mi_red_wordpress Created 1.4s
✓ Volume ejercicio_mi_wordpress_data Created 0.4s
✓ Container wordpress_db Started 11.8s
```

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress\ejerc
icio> docker-compose up -d
✓ Container wordpress_db Started 11.8s
✓ Container wordpress_phpmyadmin Started 13.6s
✓ Container wordpress_app Started 13.6s
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress\ejerc
icio> docker-compose ps
time="2026-08-20T16:29:24-05:00" level=warning msg="C:\\Users\\roble\\OneDrive\\
Desktop\\taller-docker-git\\escenario-1-wordpress\\ejercicio\\docker-compose.yml
: the attribute `version` is obsolete, it will be ignored, please remove it to a
void potential confusion"
NAME IMAGE COMMAND S
```

Escenario 1 — Ejercicio funcionando: los 3 servicios

Confirmé que phpMyAdmin (`localhost:8081`) se conecta correctamente al servidor `db` (MariaDB 10.11.18), mostrando la base de datos `wordpress` en el panel izquierdo junto con `information_schema`. WordPress (`localhost:8080`) también carga correctamente, confirmando que los tres contenedores se comunican bien a través de la red `mi_red_wordpress`.



Escenario 1 — Nota sobre la prueba de persistencia

Al reiniciar los contenedores, WordPress volvió a mostrar el instalador. Esto es esperado: la pantalla de instalación aparece siempre que WordPress no detecta una instalación completa guardada en sus tablas, independientemente de si el volumen persiste o no. Verifiqué la persistencia real con `docker volume ls`, confirmando que `mi_wordpress_data` sigue existiendo entre reinicios sin necesidad de recrearse.

Escenario 1 — Persistencia confirmada

```

PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress\ejerc
icio> docker-compose down
Desktop\\taller-docker-git\\escenario-1-wordpress\\ejercicio\\docker-compose.yml
: the attribute `version` is obsolete, it will be ignored, please remove it to a
void potential confusion"
[+] down 4/4
✓ Container wordpress_app           Removed          14.3s
✓ Container wordpress_phpmyadmin    Removed          13.3s
✓ Container wordpress_db            Removed          3.4s
✓ Network ejercicio_mi_red_wordpress Removed          0.7s
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress\ejerc
icio> docker-compose up -d

```

```

icio> docker-compose up -d
Desktop\\taller-docker-git\\escenario-1-wordpress\\ejercicio\\docker-compose.yml
: the attribute `version` is obsolete, it will be ignored, please remove it to a
void potential confusion"
[+] up 4/4
✓ Network ejercicio_mi_red_wordpress Created           0.6s
✓ Container wordpress_db           Started          5.8s
✓ Container wordpress_app           Started          7.4s
✓ Container wordpress_phpmyadmin    Started          7.5s
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress\ejerc
icio> docker volume ls

```

Al hacer `docker-compose down` y `docker-compose up -d` de nuevo, el log no mostró "Volume Created" para `mi_wordpress_data` (solo la red y los contenedores), indicando que Docker reutilizó el volumen existente en lugar de crear uno nuevo. Se confirmó también con `docker volume ls`, donde `ejercicio_mi_wordpress_data` sigue presente. Esto demuestra que los datos persisten correctamente entre reinicios, incluso si el asistente de instalación de WordPress vuelve a aparecer (eso depende del estado de las tablas, no del volumen).

```

PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress\ejerc
icio> docker volume ls
DRIVER      VOLUME NAME
local       823eccef391e223f620fe34dad5e7300a1da4a39ceb3785d617876a5bce57d3a
local       d860a7e91507a5dc4392f0b89e1b7bfb11ccf263c4037571f987c5521815dc2b
local       e470213a0087256e864f404838adbc934e7a33751c0b50a7525e8197d51f4ed8
local       ejemplo_db_data
local       ejemplo_wp_data
local       ejercicio_mi_wordpress_data
local       mi-app_datos_db
local       mi-app_datos_ej9
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress\ejerc

```

Escenario 1 — Cierre: merge a main

Con todo el escenario probado y documentado en la rama `escenario-1-wordpress`, hice el merge a `main` (`git checkout main + git merge escenario-1-wordpress`) y subí los cambios con `git push origin main`. El merge fue tipo *fast-forward*, integrando los 4 archivos del ejemplo y del ejercicio sin conflictos.

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress\ejerc  
• icio> docker-compose down  
time="2026-08-20T16:44:15-05:00" level=warning msg="C:\\Users\\roble\\OneDrive\\  
Desktop\\taller-docker-git\\escenario-1-wordpress\\ejercicio\\docker-compose.yml  
: the attribute `version` is obsolete, it will be ignored, please remove it to a  
void potential confusion"  
[+] down 4/4  
✓ Container wordpress_phpmyadmin      Removed      6.4s  
✓ Container wordpress_app             Removed      9.3s  
✓ Container wordpress_db               Removed      3.8s  
✓ Network ejercicio_mi_red_wordpress   Removed      0.7s  
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-1-wordpress\ejerc
```

```
• icio> cd ../../  
• PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git add .  
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git commit -m "feat: agreg  
• a ejercicio con MariaDB, phpMyAdmin y variables de entorno"  
[escenario-1-wordpress 142b5f4] feat: agrega ejercicio con MariaDB, phpMyAdmin y  
variables de entorno  
3 files changed, 87 insertions(+)  
create mode 100644 escenario-1-wordpress/ejercicio/.env.example  
create mode 100644 escenario-1-wordpress/ejercicio/README.md  
create mode 100644 escenario-1-wordpress/ejercicio/docker-compose.yml  
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git push origin escenario-
```

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git push origin escenario-  
• 1-wordpress  
>>  
Enumerating objects: 9, done.  
Counting objects: 100% (9/9), done.  
Delta compression using up to 4 threads  
Compressing objects: 100% (7/7), done.  
Writing objects: 100% (7/7), 1.43 KiB | 27.00 KiB/s, done.  
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)  
remote: Resolving deltas: 100% (1/1), completed with 1 local object.  
To https://github.com/robledocordoba123-cmyk/taller-docker-git.git  
505f4b8..142b5f4  escenario-1-wordpress -> escenario-1-wordpress
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git checkout main
>>
Deletion of directory 'escenario-1-wordpress/ejemplo' failed. Should I try again
? (y/n) Y
Deletion of directory 'escenario-1-wordpress/ejemplo' failed. Should I try again
? (y/n) n
Updating files: 100% (4/4), done.
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
● PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
```

```
● PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git merge escenario-1-word
● press
Updating fe96063..142b5f4
Fast-forward
escenario-1-wordpress/ejemplo/docker-compose.yml | 42 ++++++
escenario-1-wordpress/ejercicio/.env.example      | 7 +++
escenario-1-wordpress/ejercicio/README.md         | 28 ++++++
escenario-1-wordpress/ejercicio/docker-compose.yml | 52 ++++++
4 files changed, 129 insertions(+)
```

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git merge escenario-1-word
press
4 files changed, 129 insertions(+)
create mode 100644 escenario-1-wordpress/ejemplo/docker-compose.yml
create mode 100644 escenario-1-wordpress/ejercicio/.env.example
create mode 100644 escenario-1-wordpress/ejercicio/README.md
create mode 100644 escenario-1-wordpress/ejercicio/docker-compose.yml
● PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/robledocordoba123-cmyk/taller-docker-git.git
fe96063..142b5f4 main -> main
○ PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> 
```

Conclusión — Escenario 1

Este primer escenario permitió entender la lógica básica de Docker Compose: cómo varios contenedores (base de datos, aplicación, herramienta de administración) se coordinan dentro de una misma red para funcionar como un solo sistema, y cómo los volúmenes garantizan que los datos sobrevivan a reinicios o recreaciones de los contenedores.

El paso del ejemplo al ejercicio dejó claras algunas buenas prácticas concretas: separar las credenciales del código usando variables de entorno (`.env` + `.gitignore`) en vez de

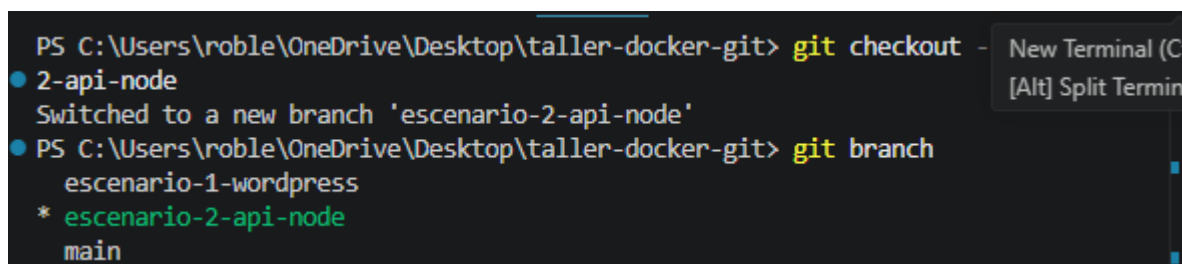
escribirlas directamente en el `docker-compose.yml`, y nombrar los recursos (volúmenes, redes) de forma descriptiva en lugar de dejar los nombres genéricos por defecto.

También surgió un obstáculo real y su solución: el error "Error establishing a database connection" al levantar los contenedores por primera vez, causado porque `depends_on` solo espera a que el contenedor de la base de datos **arranque**, no a que esté completamente listo para recibir conexiones. Se resolvió esperando unos segundos y recargando, aunque queda como aprendizaje que un `healthcheck` (que se usa en el Escenario 2) es una solución más robusta para este mismo problema.

Finalmente, se verificó la persistencia de datos comparando los logs de `docker-compose up` entre el primer y segundo levantamiento: la ausencia de la línea "Volume Created" en el segundo intento, junto con `docker volume ls`, confirmó que Docker reutilizó el volumen existente en lugar de crear uno nuevo.

Escenario 2 — Preparación

Creé la rama `escenario-2-api-node` con `git checkout -b escenario-2-api-node` y verifiqué con `git branch` que quedó activa. A diferencia del Escenario 1 —donde se usaban imágenes ya construidas como `wordpress` o `mariadb`—, en este escenario se construye una aplicación propia: primero se escribe el código de la API (`server.js`) y luego se empaqueta con un Dockerfile.

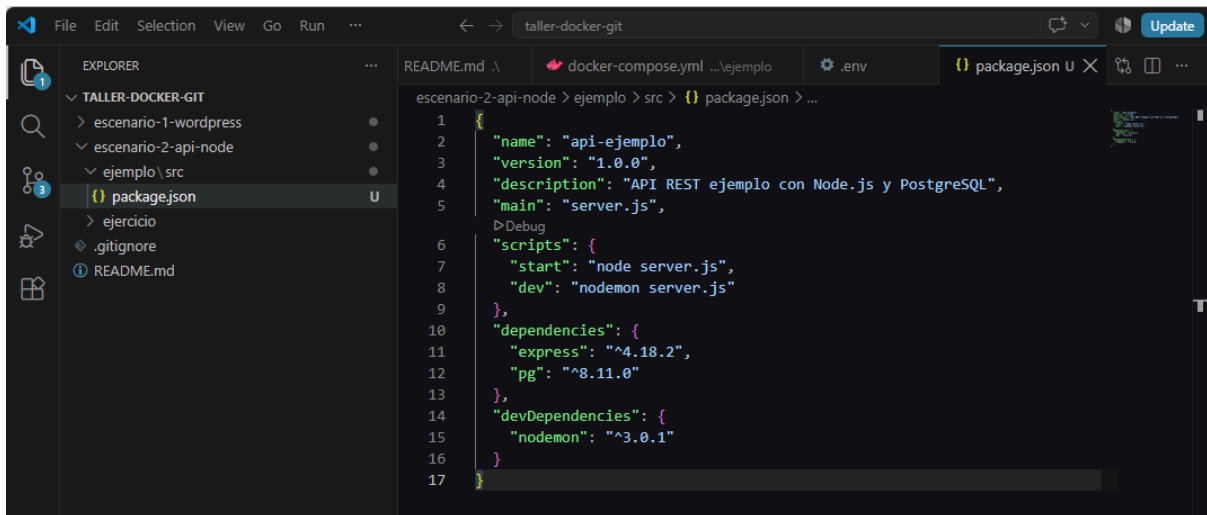


```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git checkout -b 2-api-node
Switched to a new branch 'escenario-2-api-node'
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git branch
escenario-1-wordpress
* escenario-2-api-node
main
```

Armé la estructura de carpetas dentro de `escenario-2-api-node/`: una carpeta `ejemplo/src` para el código guiado y una carpeta `ejercicio/` que se completará en la Parte B.

Configuración del proyecto (`package.json`)

En `ejemplo/src/package.json` definí las dependencias del proyecto: `express` para levantar el servidor HTTP y `pg` (node-postgres) para conectarse a PostgreSQL. Como dependencia de desarrollo agregué `nodemon`, que reinicia el servidor automáticamente cada vez que se guarda un cambio en el código, evitando tener que detenerlo y volver a correrlo manualmente.



Código de la API (**server.js**)

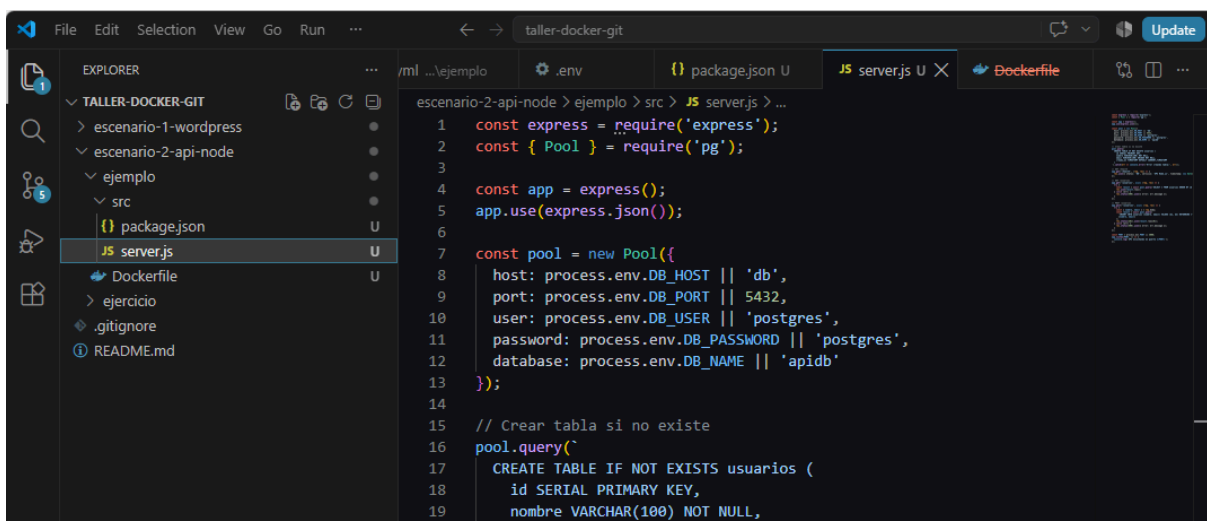
Escribí el servidor en **ejemplo/src/server.js** con Express. Los puntos clave:

- Se crea un *pool* de conexiones a PostgreSQL en vez de una sola conexión, para manejar mejor varias peticiones simultáneas.

- Los valores de conexión (host, usuario, contraseña, etc.) se leen de variables de entorno con un valor por defecto (`process.env.DB_HOST || 'db'`), donde `'db'` corresponde al nombre del servicio en el `docker-compose.yml` — así los contenedores se comunican entre sí por nombre, no por IP.

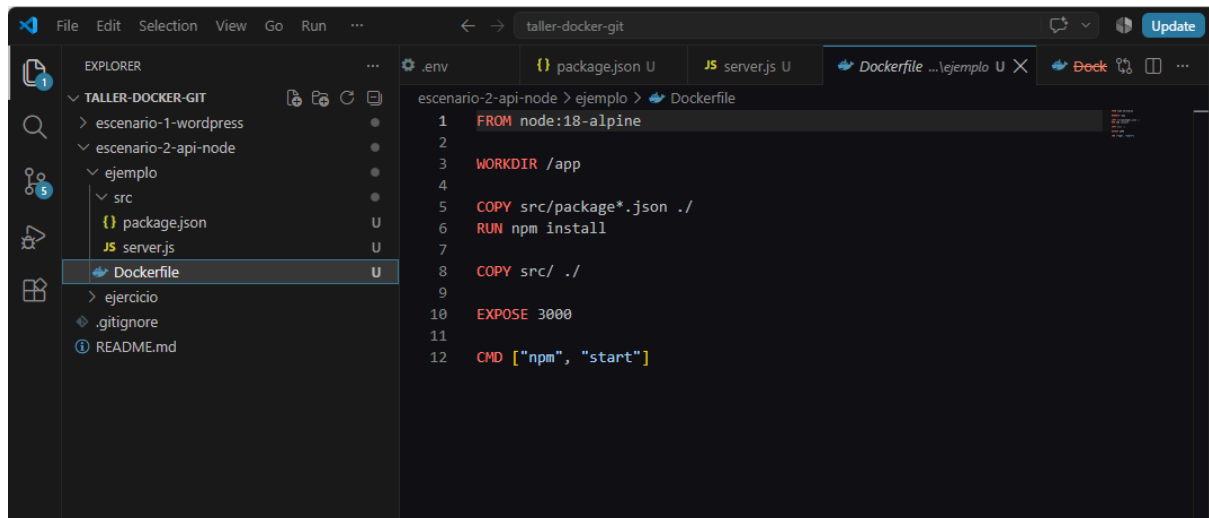
- Al arrancar, el servidor ejecuta `CREATE TABLE IF NOT EXISTS usuarios` para crear la tabla automáticamente si no existe, sin perder los datos si el contenedor se reinicia.

- Se definieron tres endpoints: `GET /health` (verificar que el servicio está vivo), `GET /usuarios` (listar) y `POST /usuarios` (crear), usando parámetros `$1`, `$2` en las consultas para evitar inyección SQL.



Dockerfile del ejemplo

Creé el `Dockerfile` en `escenario-2-api-node/ejemplo/`, usando como base la imagen `node:18-alpine` (versión liviana de Node.js sobre Alpine Linux). La buena práctica aplicada aquí es copiar primero el `package.json` e instalar las dependencias con `npm install` antes de copiar el resto del código fuente. Esto aprovecha el sistema de capas (*cache*) de Docker: si más adelante solo cambio el código de la app pero no las dependencias, Docker reutiliza la capa ya construida en vez de reinstalar todo de nuevo, ahorrando tiempo de build.



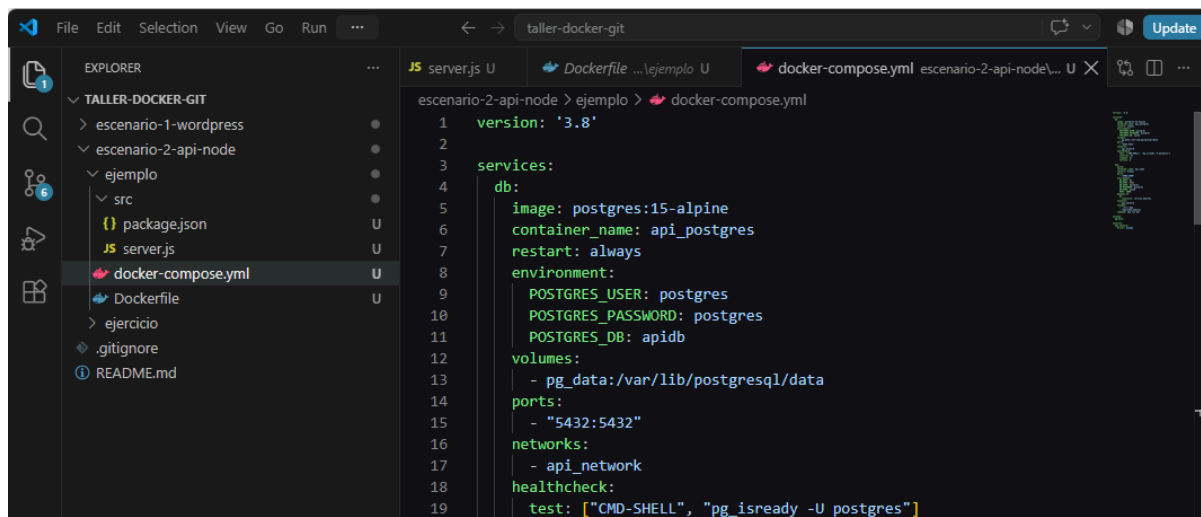
docker-compose.yml — Mejora clave: healthcheck

Creé el `docker-compose.yml` en `escenario-2-api-node/ejemplo/` con dos servicios: `db` (PostgreSQL 15) y `api` (la aplicación Node.js construida con el Dockerfile propio).

La mejora principal frente al Escenario 1 está en el servicio `db`: se agregó un `healthcheck` que ejecuta `pg_isready -U postgres` cada 5 segundos para verificar si PostgreSQL ya está realmente listo para aceptar conexiones. Combinado con `depends_on: db: condition: service_healthy` en el servicio `api`, esto hace que el contenedor de la API espere de verdad a que la base de datos esté disponible, y no solo a que el contenedor haya arrancado.

Esto resuelve de raíz el problema de "Error establishing a database connection" que se presentó en el Escenario 1, donde `depends_on` sin condición solo garantizaba el orden de arranque, pero no que el servicio estuviera listo para recibir conexiones.

También se montó el código fuente como volumen (`./src:/app`) junto con `command: npm run dev`, para que `nodemon` detecte cambios en el código sin necesidad de reconstruir la imagen en cada edición.



Levantar y probar el ejemplo

Ejecuté `docker-compose up --build -d` desde `escenario-2-api-node/ejemplo`.

En el primer intento, el contenedor `api_postgres` no alcanzó a pasar el healthcheck dentro de los reintentos configurados (5 intentos de 5 segundos) porque era la primera vez que se inicializaba la base de datos desde cero, y `api_node` no llegó a arrancar por la condición `service_healthy`. Al volver a ejecutar el comando, Postgres ya estaba inicializado y respondió a tiempo: `api_postgres` pasó a estado `Healthy` y `api_node` arrancó correctamente.

Esto confirma en la práctica el propósito del healthcheck: la API nunca intentó conectarse a una base de datos que no estuviera realmente lista, evitando el error de conexión que sí ocurrió en el Escenario 1.

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejem
plo> docker-compose up --build -d
#12 DONE 0.9s

#13 resolving provenance for metadata file
#13 DONE 0.2s
[+] up 3/3
✓ Image ejemplo-api      Built      22.2s
✓ Container api_postgres Healthy    10.2s
✓ Container api_node     Started   27.4s
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejem
plo>
```

Probando el endpoint /health

Con los contenedores arriba, probé el endpoint de salud desde la terminal:

```
plo> curl.exe http://localhost:3000/health
{"status":"OK","servicio":"API Node.js","timestamp":"2026-08-21T00:54:25.746Z"}
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejem
plo>
```

Esto confirma que la API está corriendo y respondiendo correctamente dentro del contenedor.

Probando POST /usuarios

Creé un usuario de prueba con:

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejem
plo> curl.exe -X POST http://localhost:3000/usuarios -H "Content-Type: applica
tion/json" -d '{"nombre":"Tifany","email":"tifany@test.com"}'
{"id":1,"nombre":"Tifany","email":"tifany@test.com","creado_en":"2026-08-21T00
:57:18.640Z"}
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejem
plo>
```

El registro se guardó correctamente en la tabla `usuarios` de PostgreSQL, con el `id` autoincremental (`SERIAL`) y la marca de tiempo generada automáticamente por `creado_en`.

Probando GET /usuarios

```
plo> curl.exe http://localhost:3000/usuarios
[{"id":1,"nombre":"Tifany","email":"tifany@test.com","creado_en":"2026-08-21T0
0:57:18.640Z"}]
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejem
plo>
```

Se confirma que la API lee correctamente desde PostgreSQL, devolviendo el usuario creado en el paso anterior.

Cierre — Parte A (Ejemplo guiado)

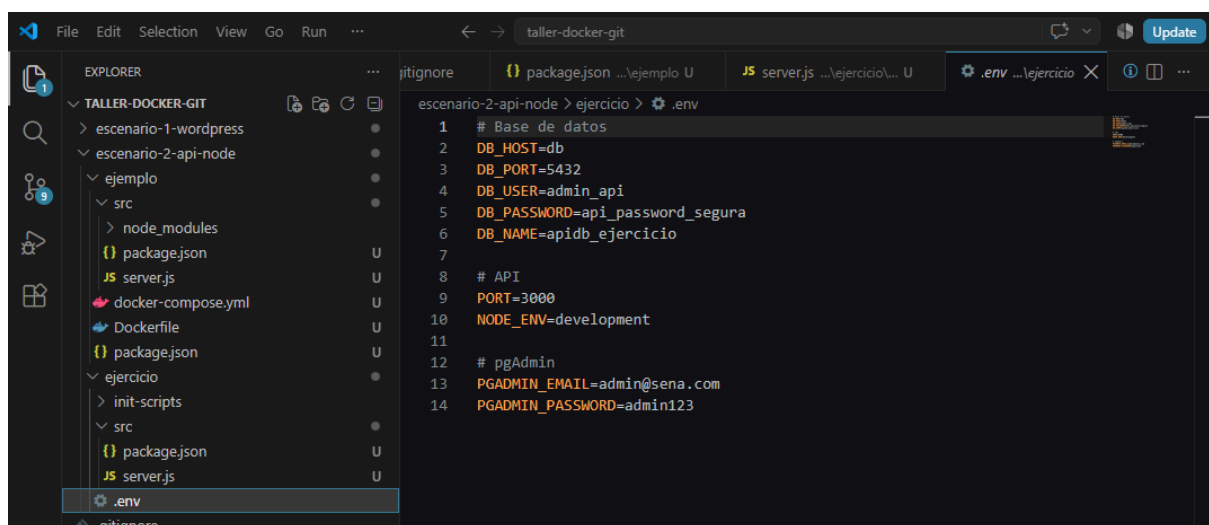
Con esto queda completo el ejemplo guiado del Escenario 2: se construyó una imagen propia con Dockerfile (a diferencia del Escenario 1, donde se usaban imágenes ya hechas), se levantó junto a PostgreSQL usando Docker Compose, y se probaron los tres endpoints (`/health`, `GET /usuarios`, `POST /usuarios`), confirmando que la API se conecta correctamente a la base de datos y persiste la información.

El punto más importante de este ejemplo fue el uso de `healthcheck + depends_on: condition: service_healthy`, que evitó el error de conexión que sí se presentó en el

Escenario 1: en el primer intento de levantar el stack, `api_postgres` no alcanzó a pasar el healthcheck a tiempo (por ser la primera inicialización de la base de datos) y `api_node` no llegó a arrancar; en el segundo intento, con la base ya inicializada, el healthcheck pasó a tiempo y ambos contenedores quedaron arriba correctamente.

Escenario 2 — Parte B: Ejercicio propio

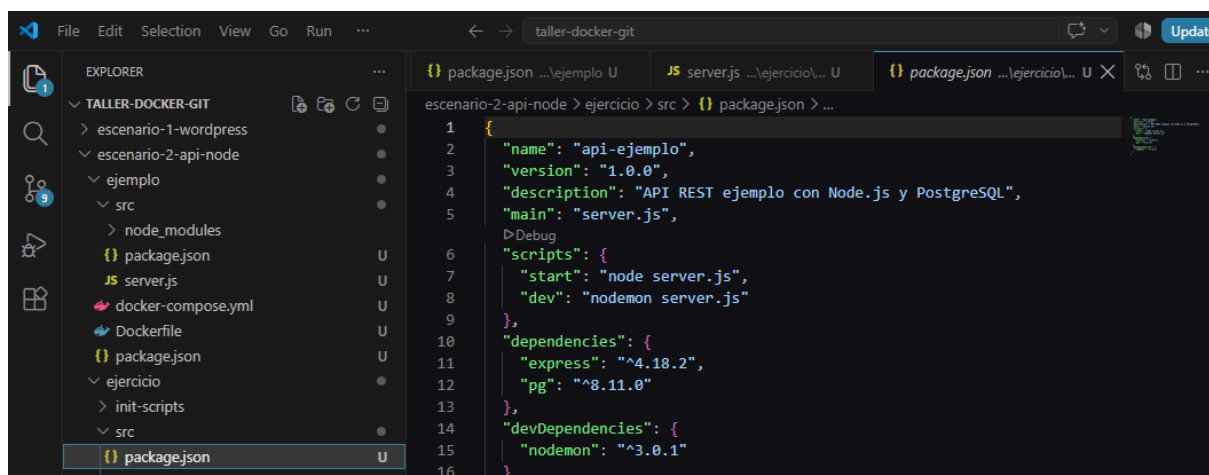
Para el ejercicio se creó la carpeta `escenario-2-api-node/ejercicio/` con estructura independiente del ejemplo guiado. Se agregó un archivo `.env` para centralizar toda la configuración (credenciales de base de datos, puerto de la API y credenciales de pgAdmin), evitando hardcodear estos valores en el código o en el `docker-compose.yml`, tal como se hizo con `.env` en el Escenario 1.



The screenshot shows the Visual Studio Code interface with the Explorer on the left and the Editor on the right. The Explorer shows a project structure with folders 'escenario-1-wordpress', 'escenario-2-api-node', and 'ejercicio'. The 'ejercicio' folder is expanded, showing files like 'package.json', 'server.js', 'docker-compose.yml', 'Dockerfile', and '.env'. The Editor displays the content of the '.env' file:

```
1 # Base de datos
2 DB_HOST=db
3 DB_PORT=5432
4 DB_USER=admin_api
5 DB_PASSWORD=api_password_segura
6 DB_NAME=apidb_ejercicio
7
8 # API
9 PORT=3000
10 NODE_ENV=development
11
12 # pgAdmin
13 PGADMIN_EMAIL=admin@sena.com
14 PGADMIN_PASSWORD=admin123
```

En el `package.json` se agregó la dependencia `dotenv`, que permite que Node.js lea el archivo `.env` y cargue sus valores como variables de entorno del proceso.

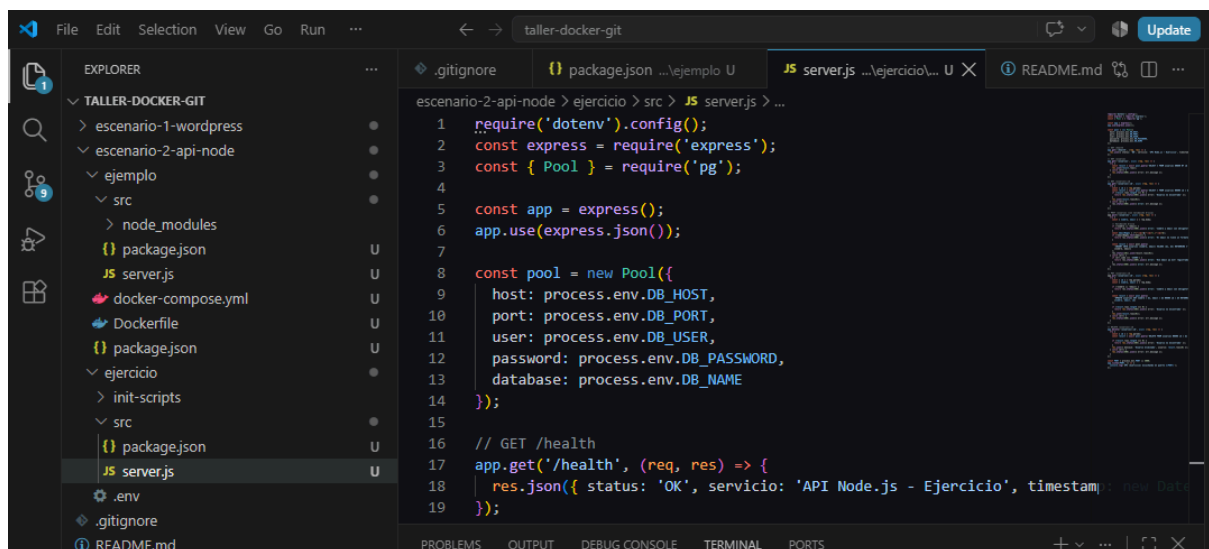


The screenshot shows the Visual Studio Code interface with the Explorer on the left and the Editor on the right. The Explorer shows the same project structure as before. The Editor displays the content of the 'package.json' file in the 'ejercicio' directory:

```
1 {
2   "name": "api-ejemplo",
3   "version": "1.0.0",
4   "description": "API REST ejemplo con Node.js y PostgreSQL",
5   "main": "server.js",
6   "scripts": {
7     "start": "node server.js",
8     "dev": "nodemon server.js"
9   },
10  "dependencies": {
11    "express": "^4.18.2",
12    "pg": "^8.11.0"
13  },
14  "devDependencies": {
15    "nodemon": "^3.0.1"
16  }
17 }
```

Mejoras en `server.js` respecto al ejemplo guiado:

- Se agregaron dos endpoints nuevos: `PUT /usuarios/:id` (actualizar un usuario) y `DELETE /usuarios/:id` (eliminar un usuario), además de un `GET /usuarios/:id` de apoyo.
- Se implementó **validación básica de datos** en `POST /usuarios`: se verifica que `nombre` y `email` no vengan vacíos y se valida el formato del correo con una expresión regular antes de intentar insertarlo en la base de datos.
- Se manejó el caso de email duplicado capturando el código de error `23505` de PostgreSQL (violación de restricción única) para devolver un mensaje claro (`409 Conflict`) en vez de un error genérico.
- A diferencia del ejemplo, aquí la tabla `usuarios` **no** se crea desde el código de la aplicación (`CREATE TABLE IF NOT EXISTS`), sino mediante un script de migración SQL que ejecuta PostgreSQL automáticamente al iniciar el contenedor por primera vez — cumpliendo el requisito de implementar *migrations*.



```
escenario-2-api-node > ejercicio > src > JS server.js > ...
1  require('dotenv').config();
2  const express = require('express');
3  const { Pool } = require('pg');
4
5  const app = express();
6  app.use(express.json());
7
8  const pool = new Pool({
9    host: process.env.DB_HOST,
10   port: process.env.DB_PORT,
11   user: process.env.DB_USER,
12   password: process.env.DB_PASSWORD,
13   database: process.env.DB_NAME
14 });
15
16 // GET /health
17 app.get('/health', (req, res) => {
18   res.json({ status: 'OK', servicio: 'API Node.js - Ejercicio', timestamp: new Date()
19 });
```

Primer intento — la API no responde

Al ejecutar `docker-compose up --build -d` por primera vez en el ejercicio, los tres contenedores quedaron en estado `Started/Healthy/Running`. Sin embargo, al probar el endpoint con `curl.exe http://localhost:3001/health` y `curl.exe http://localhost:3001/usuarios`, ambos devolvieron `Empty reply from server`, indicando que la API no estaba respondiendo correctamente.

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejerci
cio> docker-compose up --build -d
[+] up 4/4
✓ Image ejercicio-api Built 113.1s
✓ Container ejercicio_postgres Healthy 22.5s
✓ Container ejercicio_pgadmin Running 0.0s
✓ Container ejercicio_api Started 25.1s
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejerci
cio> curl.exe http://localhost:3001/health
curl: (52) Empty reply from server
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejerci
cio> curl.exe http://localhost:3001/usuarios
```

Diagnóstico — revisando contenedores activos

Para descartar que el contenedor se hubiera caído, verifiqué el estado de todos los contenedores con `docker ps`, confirmando que `ejercicio_api` seguía "Up" y corriendo.

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejerci
cio> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
7cd0cfcb2ad    dpag/pgadmin4:latest               "/entrypoint.sh"        2 minutes ago Up About a minute 0.0.0.0:5050->80/tcp, [::]:5050->80/tcp
ejercicio_pgadmin
b0383e4923c5   ejercicio-api                       "docker-entrypoint.s..." 2 minutes ago Up About a minute 0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp
ejercicio_api
b87f05ae64f4   postgres:15-alpine                "docker-entrypoint.s..." 2 minutes ago Up About a minute
```

Diagnóstico — revisando logs del contenedor

Con `docker logs ejercicio_api` encontré el error real: `Error: Cannot find module 'dotenv'`. El proceso de Node.js se estaba cayendo internamente (nodemon lo reiniciaba en bucle) porque no encontraba esa dependencia, aunque el `server.js` la requería desde la primera línea.

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejerci
cio> docker logs ejercicio_api

> api-ejemplo@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
node:internal/modules/cjs/loader:1143
```

Causa raíz — package.json incorrecto

Verifiqué el contenido de `ejercicio/src/package.json` con `type` y encontré que, por un cruce de archivos durante la organización de carpetas, ese archivo tenía el contenido del **ejemplo** (`"name": "api-ejemplo"`, sin la dependencia `dotenv`), en vez del `package.json` propio del ejercicio.

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejerci
cio> type ..\ejercicio\src\package.json
{
  "name": "api-ejemplo",
  "version": "1.0.0",
  "description": "API REST ejemplo con Node.js y PostgreSQL",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
```

Corrección — package.json y rebuild

Reescribí `ejercicio/src/package.json` con el contenido correcto (incluyendo `"dotenv": "^16.3.1"`) y ejecuté `docker-compose up --build -d` para reconstruir la imagen con la dependencia correcta instalada.

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejerci
cio> type src\package.json
{
  "name": "api-ejercicio",
  "version": "1.0.0",
  "description": "API REST con Node.js y PostgreSQL - Ejercicio",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
```

Segundo obstáculo — volumen anónimo desactualizado

A pesar del rebuild, el error de `dotenv` persistió. La causa fue que `docker-compose.yml` declara un volumen anónimo (`- /app/node_modules`) que Docker reutiliza entre reconstrucciones. Ese volumen se había creado en el primer intento fallido (sin `dotenv`), y seguía sobrescribiendo el `node_modules` correcto de la imagen recién construida.

```

PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejerci
cio> docker-compose up --build -d
[+] up 4/4
✓ Image ejercicio-api Built 113.1s
✓ Container ejercicio_postgres Healthy 22.5s
✓ Container ejercicio_pgadmin Running 0.0s
✓ Container ejercicio_api Started 25.1s
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejerci
cio> curl.exe http://localhost:3001/health
curl: (52) Empty reply from server
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejerci
cio> curl.exe http://localhost:3001/usuarios

```

Solución — down -v y nuevo build

Ejecuté `docker-compose down -v` para eliminar también los volúmenes (no solo los contenedores), y luego `docker-compose up --build -d` para levantar todo desde cero. Esto forzó la creación de un volumen limpio, sincronizado con la imagen recién construida.

```

PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejerci
cio> docker-compose down -v
time="2026-08-20T21:09:33-05:00" level=warning msg="C:\\Users\\roble\\OneDrive\\
Desktop\\taller-docker-git\\escenario-2-api-node\\ejercicio\\docker-compose.yml:
the attribute `version` is obsolete, it will be ignored, please remove it to av
oid potential confusion"
[+] down 5/5
✓ Container ejercicio_pgadmin Removed 12.8s
✓ Container ejercicio_api Removed 10.4s
✓ Container ejercicio_postgres Removed 1.2s
✓ Network ejercicio_ejercicio_network Removed 0.7s
✓ Volume ejercicio_pg_data_ejercicio Removed 0.7s

```

```

PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejerci
cio> docker-compose up --build -d
Desktop\\taller-docker-git\\escenario-2-api-node\\ejercicio\\docker-compose.yml:
the attribute `version` is obsolete, it will be ignored, please remove it to av
oid potential confusion"
#1 [internal] load local bake definitions
#1 reading from stdin 619B 0.1s done
#1 DONE 0.1s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile:
#2 transferring dockerfile: 175B 0.7s done

```

Verificación final — la API responde correctamente

Con los contenedores ya sincronizados, probé de nuevo los endpoints:

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejercicio> curl.exe http://localhost:3001/health
>> curl.exe http://localhost:3001/usuarios
{"status":"OK","servicio":"API Node.js - Ejercicio","timestamp":"2026-08-21T02:12:16.248Z"}[{"id":1,"nombre":"Admin SENA","email":"admin@sena.edu.co","creado_en":"2026-08-21T02:11:04.009Z"}]
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejercicio>
```

Esto confirma dos cosas: que la API ya está sana, y que el script de migración `01-init.sql` se ejecutó automáticamente al inicializar PostgreSQL por primera vez (el usuario "Admin SENA" ya existía sin haberlo creado manualmente).

Probando PUT /usuarios/:id

Al probar la actualización con `curl.exe` usando comillas escapadas (`\`), PowerShell rompía el comando cuando el texto contenía espacios (por ejemplo "Admin SENA Editado"), generando errores como `Could not resolve host` o `SyntaxError: Unexpected token`. Esto se debe a cómo PowerShell interpreta las comillas al pasarlas a un programa externo como `curl.exe`, no a un error en la API.

La solución fue usar el comando nativo de PowerShell `Invoke-RestMethod`, que maneja el body JSON directamente sin conflictos de comillas:

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejercicio> curl.exe -X PUT http://localhost:3001/usuarios/1 -H "Content-Type: application/json" -d '{"nombre":"Admin SENA Editado","email":"admin@sena.edu.co"}'
7:7)<br> &nbsp; &nbsp; &nbsp;at IncomingMessage.onEnd (/app/node_modules/raw-body/index.js:287:7)<br> &nbsp; &nbsp; &nbsp;at IncomingMessage.emit (node:events:517:28)<br> &nbsp; &nbsp; &nbsp;at endReadableNT (node:internal/streams/readable:1400:12)<br> &nbsp; &nbsp; &nbsp;at process.processTicksAndRejections (node:internal/process/task_queues:82:21)</pre>
</body>
</html>
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-2-api-node\ejercicio>
```

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> Invoke-RestMethod -Uri "http://localhost:3001/usuarios/1" -Method PUT -ContentType "application/json" -Body '{"nombre":"Admin SENA Editado","email":"admin@sena.edu.co"}'

id nombre          email              creado_en
----
1 Admin SENA Editado admin@sena.edu.co 2026-08-21T02:11:04.009Z

PS C:\Users\roble\OneDrive\Desktop\taller-docker-git>
```

El usuario `id=1` quedó actualizado correctamente en PostgreSQL.

Probando POST /usuarios

Se creó un segundo usuario, específicamente para probar luego el endpoint de eliminación:

```
Invoke-RestMethod -Uri "http://localhost:3001/usuarios" -Method POST -ContentType "application/json" -Body '{"nombre":"Para Borrar","email":"borrar@test.com"}'

id nombre          email              creado_en
----
2 Para Borrar borrar@test.com 2026-08-21T02:44:14.441Z
```

Probando DELETE /usuarios/:id

Se eliminó el usuario de prueba creado en el paso anterior:

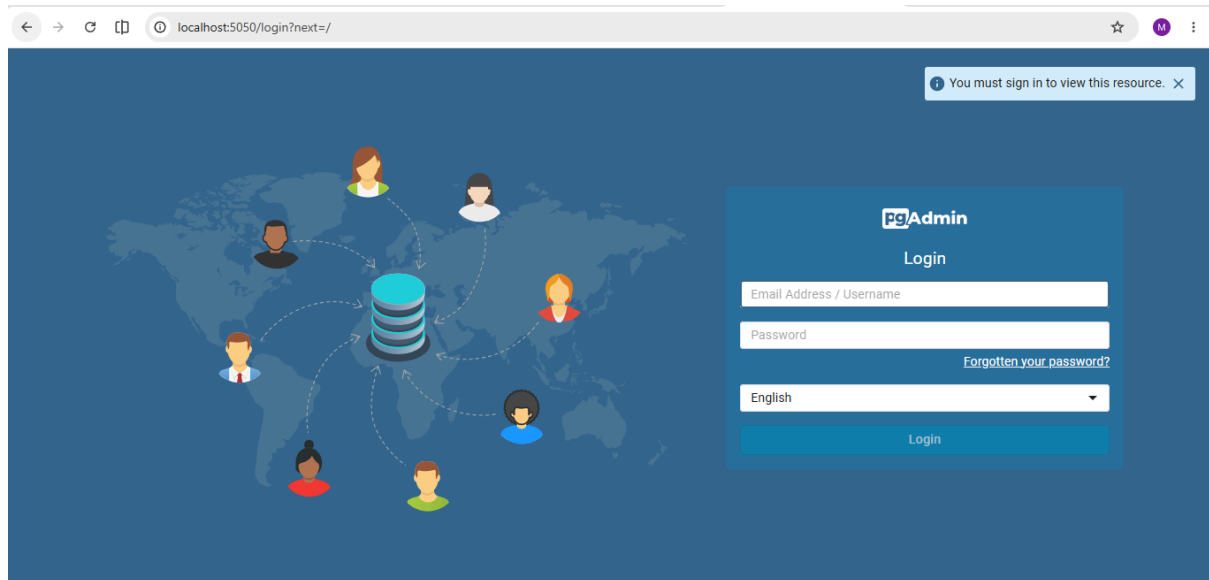
```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> Invoke-RestMethod -Uri "http://localhost:3001/usuarios/2" -Method DELETE

mensaje          usuario
-----
Usuario eliminado @id=2; nombre=Para Borrar; email=borrar@test.com; creado_en=2...
```

El endpoint confirma la eliminación devolviendo un mensaje junto con los datos del usuario borrado, tal como se definió en `server.js`. Con esto queda comprobado el CRUD completo de la API: `GET`, `POST`, `PUT` y `DELETE`.

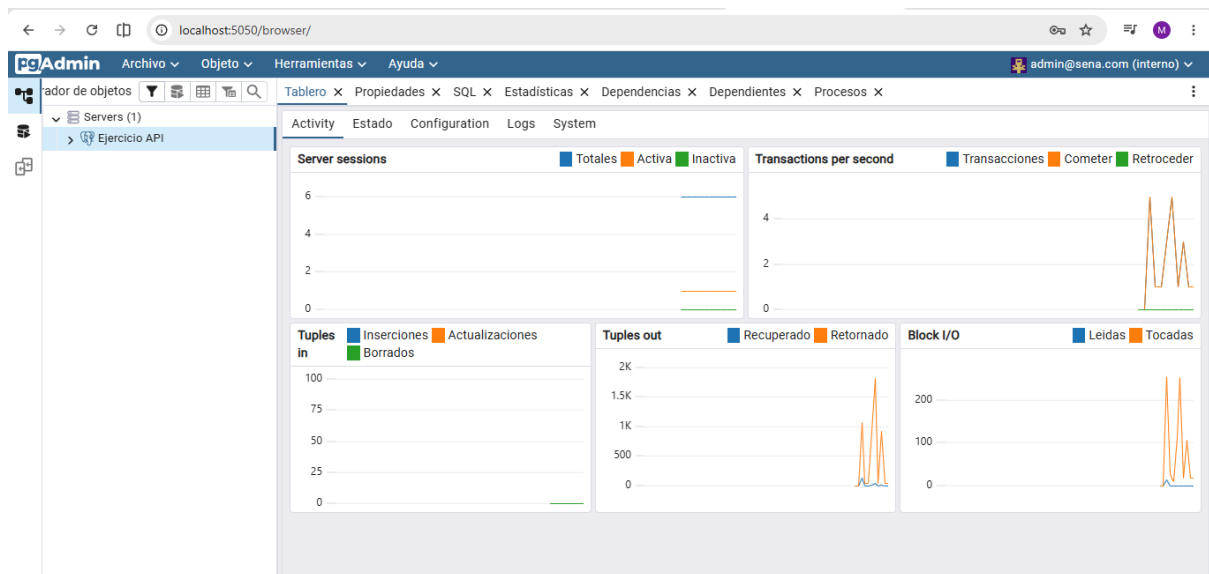
Verificando pgAdmin (puerto 5050)

Accedí a <http://localhost:5050> y confirmé que el servicio de pgAdmin quedó levantado correctamente. Inicié sesión con las credenciales configuradas en el `.env` (`PGADMIN_EMAIL` y `PGADMIN_PASSWORD`), y el panel de administración cargó sin problemas, confirmando la sesión activa como `admin@sena.com`.



Conectando pgAdmin a PostgreSQL

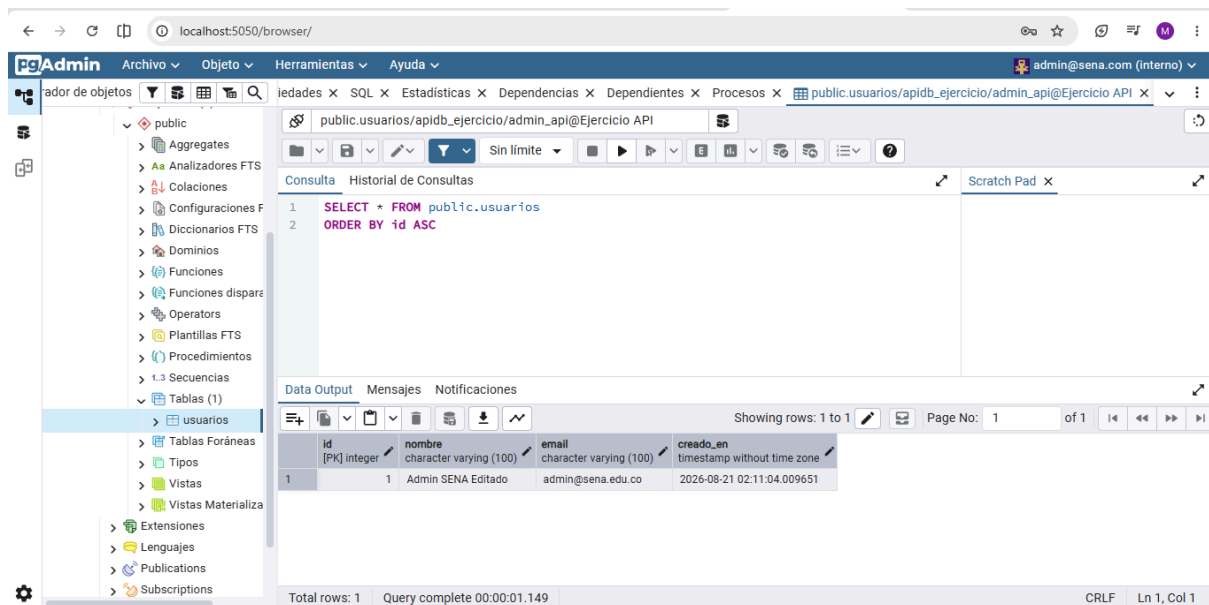
Agregué un nuevo servidor en pgAdmin con nombre "Ejercicio API", conectándome a la base de datos usando `db` como host (el nombre del servicio dentro de la red de Docker, no `localhost`, ya que pgAdmin corre en su propio contenedor dentro de la misma red `ejercicio_network`), puerto `5432`, y las credenciales definidas en el `.env`. La conexión fue exitosa y se puede ver el dashboard de actividad del servidor PostgreSQL en tiempo real.



Visualizando la tabla usuarios en pgAdmin

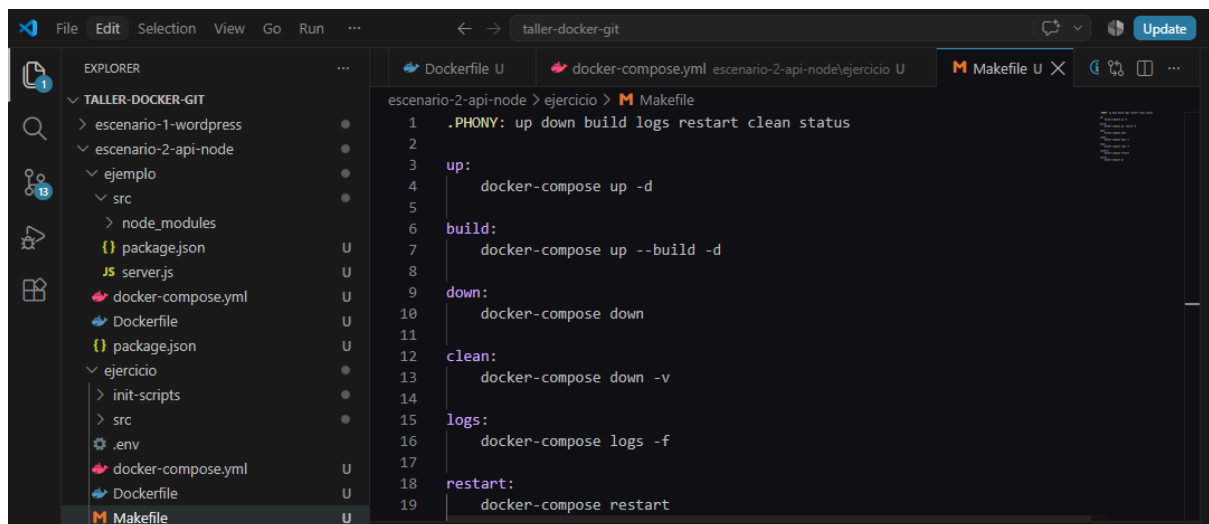
Navegando por el árbol de objetos (`Ejercicio API > Bases de Datos > apidb_ejercicio > Esquemas > public > Tablas > usuarios`), se accedió a los datos de la tabla mediante **Ver/Editar Datos** → **Todas las Filas**. pgAdmin generó automáticamente la consulta `SELECT * FROM public.usuarios ORDER BY id ASC`, mostrando el registro actualizado: `id=1, "Admin SENA Editado", admin@sena.edu.co`. El usuario con `id=2` ("Para Borrar") ya no aparece, confirmando visualmente que el `DELETE` probado anteriormente se aplicó correctamente sobre la base de datos.

Esto demuestra que pgAdmin está correctamente conectado y sincronizado con la base de datos usada por la API, cumpliendo el requisito de administración visual pedido en el ejercicio.



Makefile

Se creó un **Makefile** en `escenario-2-api-node/ejercicio/` con atajos para los comandos de Docker Compose usados con más frecuencia durante el desarrollo (**up**, **build**, **down**, **clean**, **logs**, **restart**, **status**). Esto simplifica la operación del stack: en vez de recordar y escribir comandos largos como `docker-compose up --build -d`, basta con ejecutar `make build`.



Conclusión — Escenario 2

Este escenario permitió dar un paso más allá del Escenario 1: en vez de usar únicamente imágenes ya construidas, se creó una aplicación propia (API REST con Node.js y Express) empaquetada con un Dockerfile propio, aplicando la buena práctica de copiar primero las dependencias e instalarlas antes de copiar el resto del código, para aprovechar el cache de capas de Docker.

La mejora técnica más importante fue el uso de `healthcheck` combinado con `depends_on: condition: service_healthy`, que resolvió de raíz el problema de conexión prematura que se había presentado en el Escenario 1: ahora la API espera realmente a que PostgreSQL esté lista para aceptar conexiones, no solo a que el contenedor haya arrancado.

En la Parte B se implementó un CRUD completo (`GET`, `POST`, `PUT`, `DELETE`) con validación básica de datos, manejo de errores (incluyendo el caso de email duplicado), variables de entorno centralizadas en un `.env`, migraciones automáticas mediante un script SQL ejecutado por PostgreSQL al iniciar, y una interfaz visual de administración con pgAdmin.

También surgió un obstáculo real de aprendizaje: un volumen anónimo de Docker (`/app/node_modules`) quedó desincronizado con el `package.json` tras una reconstrucción de la imagen, causando el error `Cannot find module 'dotenv'` a pesar de que el código y las dependencias declaradas eran correctos. Esto enseñó que los volúmenes anónimos pueden "atrapar" una versión antigua de las dependencias, y que `docker-compose down -v` es necesario (no solo `down`) cuando se sospecha de este tipo de inconsistencia.

Escenario 2 — Cierre: merge a main

Con todo el escenario probado y documentado en la rama `escenario-2-api-node`, se hizo el commit (`git commit -m "feat: agrega API REST con Node.js y PostgreSQL (Escenario 2)"`), el push (`git push origin escenario-2-api-node`), y finalmente el merge a `main` (`git checkout main + git merge escenario-2-api-node`) seguido de `git push origin main`. El merge fue tipo *fast-forward*, integrando los 10 archivos del ejemplo y del ejercicio sin conflictos.

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git merge escenario-2-api-node

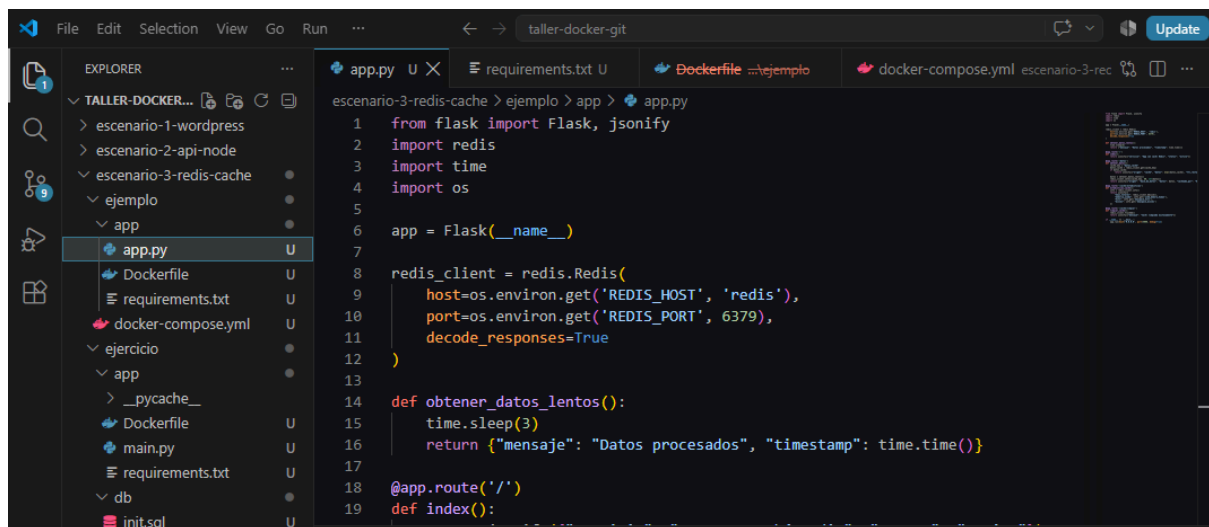
Updating 142b5f4..b83bccc
Fast-forward
 escenario-2-api-node/ejemplo/Dockerfile      | 12 +++
 escenario-2-api-node/ejemplo/docker-compose.yml | 52 ++++++++
 escenario-2-api-node/ejemplo/src/package.json | 17 +++
 escenario-2-api-node/ejemplo/src/server.js    | 57 ++++++++
 escenario-2-api-node/ejercicio/Dockerfile     | 12 +++
 escenario-2-api-node/ejercicio/Makefile       | 22 +++
 escenario-2-api-node/ejercicio/docker-compose.yml | 66 ++++++++
```

```
create mode 100644 escenario-2-api-node/ejercicio/src/server.js
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/robledocordoba123-cmyk/taller-docker-git.git
 142b5f4..b83bccc main -> main
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> 
```

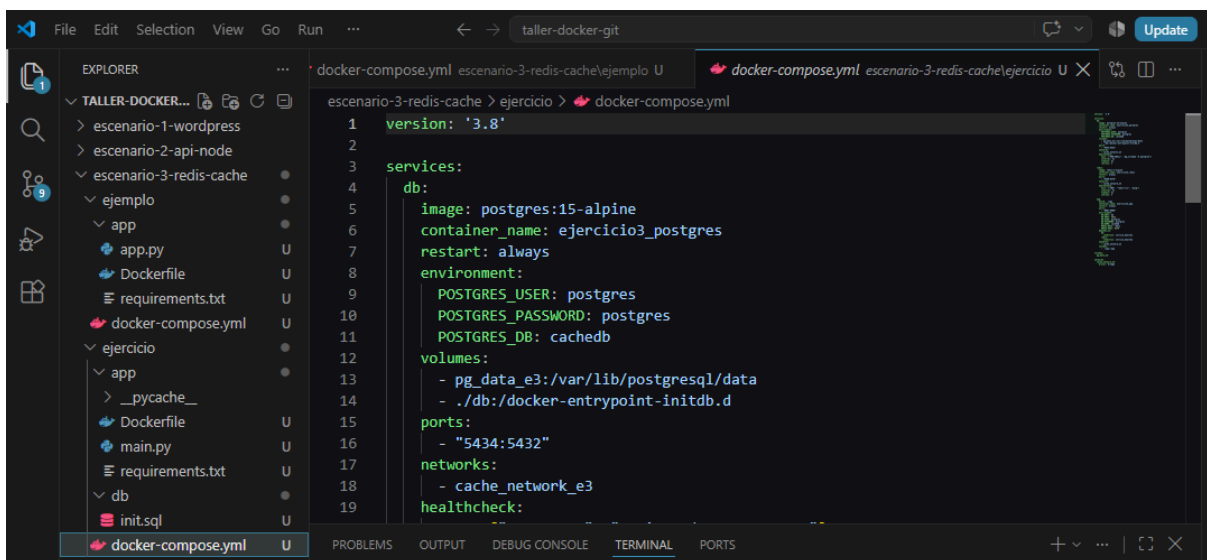
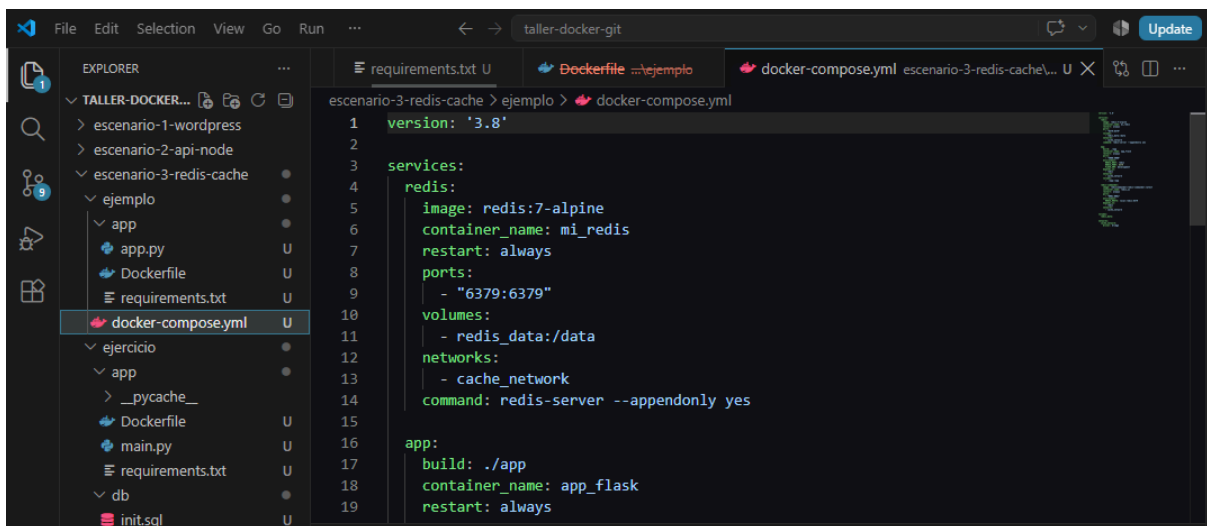
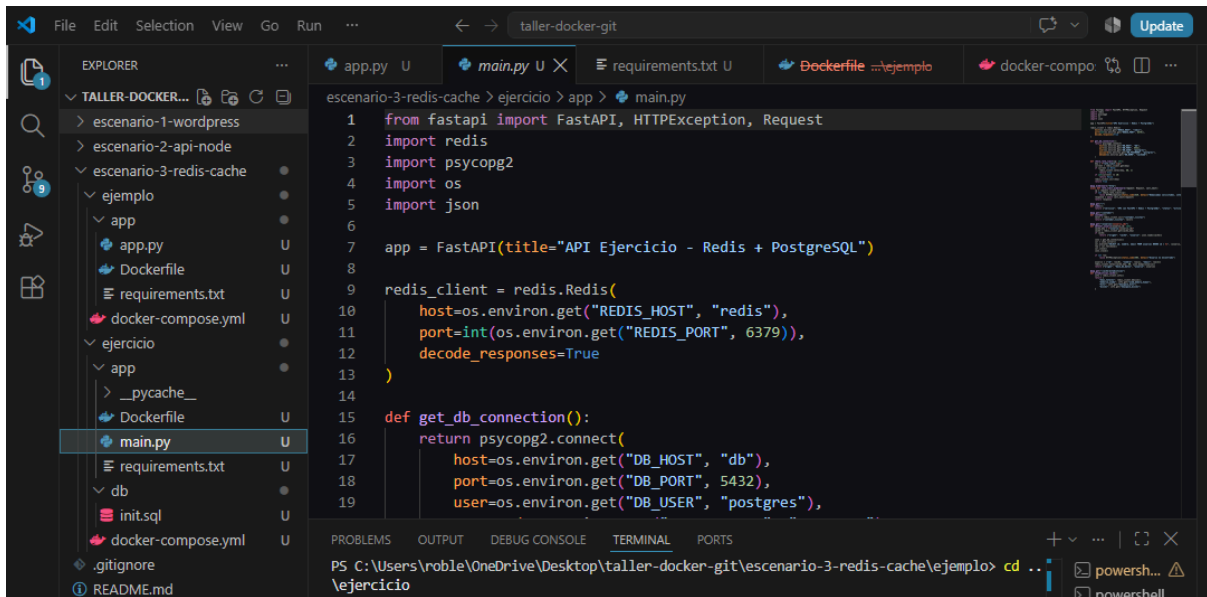
Escenario 3 — Preparación y código

Se creó la rama `escenario-3-redis-cache` con la estructura `ejemplo/` y `ejercicio/`. El ejemplo guiado usa Flask + Redis como capa de caché: el endpoint `/datos` simula una operación lenta (3 segundos) y guarda el resultado en Redis por 60 segundos con `setex`, de modo que las siguientes peticiones dentro de ese tiempo se sirven directo desde caché en vez de repetir el proceso lento. Se incluyó también Redis Commander como interfaz visual de administración (puerto 8081).

Para el ejercicio se cumplieron los requisitos de la guía: se cambió Flask por **FastAPI**, se agregó un endpoint `/contador` que usa **INCR** de Redis como contador de visitas, se implementó **rate limiting** (máximo 10 solicitudes por minuto por IP) mediante un middleware que usa Redis para llevar el conteo por IP, y se configuró **PostgreSQL como fuente de verdad** con **Redis como caché**: el endpoint `/usuarios/{id}` primero busca en Redis, y si no encuentra el dato, lo busca en PostgreSQL y lo guarda en caché para la siguiente consulta. La tabla `usuarios` se pobló automáticamente mediante un script de migración (`init.sql`) montado en `/docker-entrypoint-initdb.d`, igual que en el Escenario 2. Se agregaron **healthchecks** tanto para PostgreSQL como para Redis, y `depends_on: condition: service_healthy` en la app para ambos servicios.



```
1 from flask import Flask, jsonify
2 import redis
3 import time
4 import os
5
6 app = Flask(__name__)
7
8 redis_client = redis.Redis(
9     host=os.environ.get('REDIS_HOST', 'redis'),
10    port=os.environ.get('REDIS_PORT', 6379),
11    decode_responses=True
12 )
13
14 def obtener_datos_lentos():
15     time.sleep(3)
16     return {"mensaje": "Datos procesados", "timestamp": time.time()}
17
18 @app.route('/')
19 def index():
```



Levantar y probar ambos servicios

Se levantaron ambos stacks con `docker-compose up --build -d`. En el ejemplo, la primera petición a `/datos` tardó los 3 segundos simulados y devolvió `"origen": "base_de_datos"`; la segunda petición fue instantánea con `"origen": "cache"` y mostró el TTL restante. El endpoint `/cache/estadisticas` confirmó numéricamente el comportamiento: 2 aciertos (hits) y 1 fallo (miss) de caché.

En el ejercicio, `/contador` incrementó correctamente usando `INCR` de Redis (1, luego 2). El endpoint `/usuarios/1` mostró el mismo patrón: la primera consulta llegó a PostgreSQL (`"origen": "base_de_datos"`, con los datos de "Ana Torres" insertados por la migración) y la segunda se sirvió desde Redis (`"origen": "cache"`), confirmando que la combinación PostgreSQL + Redis como caché funciona correctamente.

```
● r-compose up --build -d
time="2026-08-20T22:52:48-05:00" level=warning msg="C:\\Users\\roble\\OneDrive\\Desktop\\taller-docker-git\\escenario-3-redis-cache\\ejemplo\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 15/16
  ✓ Image rediscommander/redis-commander:latest Pulled 70.8s
  ✓ Image redis:7-alpine Pulled 53.0s
  - Image ejemplo-app Building 0.0s
#1 [internal] load local bake definitions
#1 reading from stdin 627B 0.3s done
```

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-3-redis-cache\ejemplo> cd ..
\ejercicio
>> docker-compose up --build -d

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 241B 0.1s done
#2 DONE 0.1s

#3 [internal] load metadata for docker.io/library/python:3.11-slim
#3 DONE 1.2s

#4 [internal] load .dockerignore
```

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-3-redis-cache\ejercicio> cd ..
\ejercicio
>> docker-compose up --build -d
[+] up 7/7ejercicio_pg_data_e3 Created 0.2s
  ✓ Image ejercicio-app Built 105.2s
  ✓ Network ejercicio_cache_network_e3 Created 0.8s
  ✓ Volume ejercicio_pg_data_e3 Created 0.2s
  ✓ Container ejercicio_postgres Recreated 11.7s
  ✓ Container ejercicio3_redis Healthy 29.7s
  ✓ Container ejercicio3_app Started 31.7s
  ✓ Container ejercicio3_postgres Healthy 21.6s
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-3-redis-cache\ejercicio> cur
```

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-3-redis-cache\ejemplo> cd ..  
\\ejercicio  
>> docker-compose up --build -d  
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-3-redis-cache\ejercicio> curl  
l.exe http://localhost:5000/  
>> curl.exe http://localhost:5000/datos  
>> curl.exe http://localhost:5000/datos  
● >> curl.exe http://localhost:5000/cache/estadisticas  
{  
  "servicio": "App con cach\u00e9 Redis",  
  "status": "activo"  
}
```

```
>> curl.exe http://localhost:5000/datos ...  
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-3-redis-cache\ejercicio> cur  
● l.exe http://localhost:8001/  
>> curl.exe http://localhost:8001/contador  
>> curl.exe http://localhost:8001/contador  
>> curl.exe http://localhost:8001/usuarios/1  
>> curl.exe http://localhost:8001/usuarios/1  
{"servicio":"API con FastAPI + Redis + PostgreSQL","status":"activo"}{"contador_visitas":1}  
{"contador_visitas":2}{ "origen":"base_de_datos","usuario":{"id":1,"nombre":"Ana Torres","em  
ail":"ana@sena.edu.co"}}{"origen":"cache","usuario":{"id":1,"nombre":"Ana Torres","email":  
ana@sena.edu.co"}}  
○ PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-3-redis-cache\ejercicio> |
```

Escenario 3 — Cierre: merge a main

Con el ejemplo y el ejercicio probados, se hizo commit (feat: agrega app con cache Redis y PostgreSQL), push a la rama, y merge fast-forward a main (9 archivos, 280 inserciones), sin conflictos.

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-3-redis-cache\ejercicio> cd  
C:\Users\roble\OneDrive\Desktop\taller-docker-git  
>> git add escenario-3-redis-cache/  
>> git status  
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git commit -m "feat: agrega app con c  
ache Redis y PostgreSQL (Escenario 3)"  
>> git push origin escenario-3-redis-cache  
>> git checkout main  
>> git merge escenario-3-redis-cache  
● >> git push origin main  
[escenario-3-redis-cache f7f8b32] feat: agrega app con cache Redis y PostgreSQL (Escenario  
3)
```

```

PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git commit -m "feat: agrega app con c
ache Redis y PostgreSQL (Escenario 3)"
>> git push origin escenario-3-redis-cache
>> git checkout main ...
escenario-3-redis-cache/ejemplo/app/app.py | 49 ++++++++
.../ejemplo/app/requirements.txt | 2 +
escenario-3-redis-cache/ejemplo/docker-compose.yml | 51 ++++++++
escenario-3-redis-cache/ejercicio/app/Dockerfile | 7 ++
escenario-3-redis-cache/ejercicio/app/main.py | 81 ++++++++
.../ejercicio/app/requirements.txt | 5 ++
escenario-3-redis-cache/ejercicio/db/init.sql | 10 +++
.../ejercicio/docker-compose.yml | 68 ++++++++

```

```

PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git commit -m "feat: agrega app con c
ache Redis y PostgreSQL (Escenario 3)"
>> git push origin escenario-3-redis-cache
>> git checkout main ...
To https://github.com/robledocordoba123-cmyk/taller-docker-git.git
b83bcc..f7f8b32 main -> main
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git status
● On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
○ PS C:\Users\roble\OneDrive\Desktop\taller-docker-git>

```

Evidencia del rate limiting -Escenario 3

```

>> }
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-3-redis-cache\ejercicio> for ($i=1; $i -le 12; $i++) {
>> try {
>> $r = Invoke-WebRequest -Uri "http://localhost:8001/" -UseBasicParsing
>> Write-Host "Request $i : $($r.StatusCode)"
● >> } catch {
>> $codigo = $_.Exception.Response.StatusCode.value__
>> Write-Host "Request $i : $codigo"
>> }
>> }
Request 1 : 200
Request 2 : 200

```

```

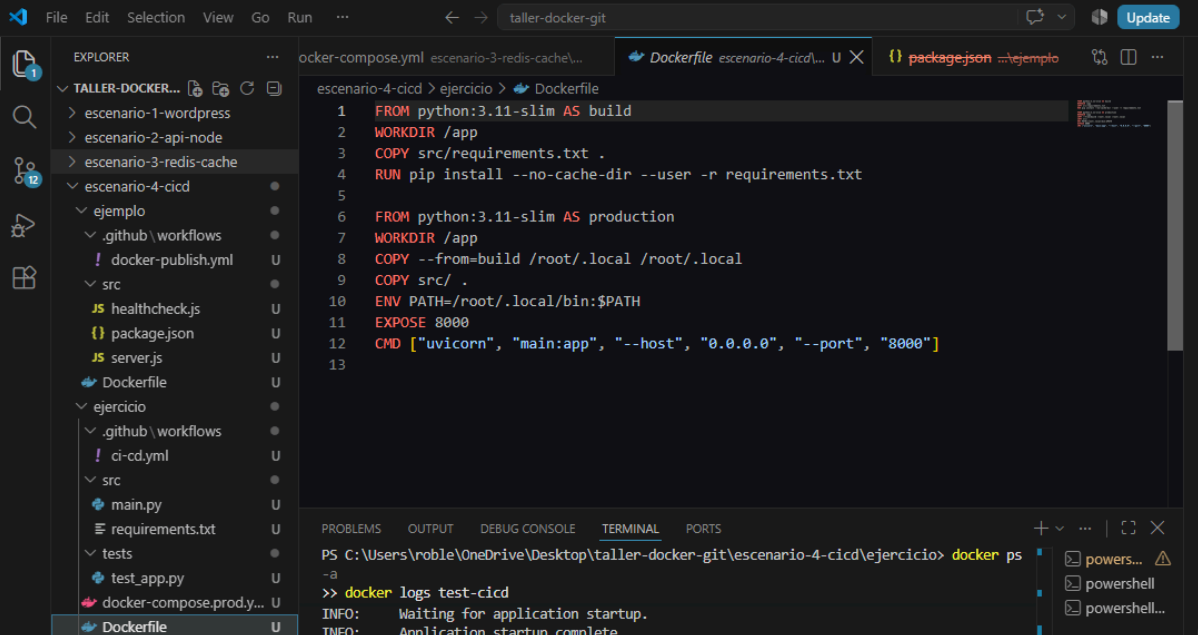
...
Request 6 : 200
Request 7 : 200
Request 8 : 200
Request 9 : 200
Request 10 : 200
Request 11 : 429
Request 12 : 429
○ PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-3-redis-cache\ejercicio>

```

Escenario 4 — Preparación y código

Se creó la rama `escenario-4-cicd`. El ejemplo guiado usa Node.js con Express, empaquetado en una imagen Docker con `HEALTHCHECK` propio (`healthcheck.js`), y un workflow de GitHub Actions (`docker-publish.yml`) que construye y publica la imagen en DockerHub usando `docker/build-push-action`, con tags automáticos generados según la rama, versión semántica, o `latest` en la rama principal.

Para el ejercicio se implementaron todos los requisitos adicionales de la guía: la aplicación se reescribió en **FastAPI** (Python) con pruebas unitarias (`tests/test_app.py` usando `pytest` y `TestClient`); el Dockerfile usa **multi-stage build** (una etapa `build` para instalar dependencias y otra `production` más liviana que solo copia lo necesario para ejecutar); el workflow (`ci-cd.yml`) ejecuta primero los tests, y solo si pasan (`needs: test`) procede a construir y publicar la imagen; la publicación se hace tanto en **DockerHub** como en **GitHub Container Registry (GHCR)**; se agregó un paso de **escaneo de vulnerabilidades con Trivy** sobre la imagen construida; y se incluyó un `docker-compose.prod.yml` que consume la imagen ya publicada en lugar de construirla localmente.



The screenshot shows a Visual Studio Code editor window with the following components:

- EXPLORER:** A file tree on the left showing the project structure. The 'ejercicio' folder is expanded, showing files like `main.py`, `requirements.txt`, `test_app.py`, and `docker-compose.prod.yml`. The `Dockerfile` is selected.
- EDITOR:** The main workspace displays the content of the `Dockerfile`. It uses a multi-stage build with two stages: `build` and `production`. The `build` stage installs dependencies from `requirements.txt`. The `production` stage copies the necessary files to a smaller image and sets the command to run `uvicorn`.
- TERMINAL:** A terminal window at the bottom shows the command `docker ps` being executed. The output indicates that the container is running and the application startup is complete.

```
1 FROM python:3.11-slim AS build
2 WORKDIR /app
3 COPY src/requirements.txt .
4 RUN pip install --no-cache-dir --user -r requirements.txt
5
6 FROM python:3.11-slim AS production
7 WORKDIR /app
8 COPY --from=build /root/.local /root/.local
9 COPY src/ .
10 ENV PATH=/root/.local/bin:$PATH
11 EXPOSE 8000
12 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
13
```

PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-4-cid\ejercicio> docker ps
-a
>> docker logs test-cid
INFO: Waiting for application startup.
INFO: Application startup complete.

This screenshot shows the Visual Studio Code interface with the file explorer on the left displaying a project structure for 'TALLER-DOCKER-GIT'. The main editor window is open to the file 'docker-publish.yml' in the path 'escenario-4-cicd > ejemplo > .github > workflows'. The code defines a GitHub Actions workflow named 'Build y Push a DockerHub'.

```
1 name: Build y Push a DockerHub
2
3 on:
4   push:
5     branches: [ main, develop ]
6     tags: [ "v*" ]
7   pull_request:
8     branches: [ main ]
9
10 env:
11   REGISTRY: docker.io
12   IMAGE_NAME: usuario-dockerhub/app-cicd
13
14 jobs:
15   build:
16     runs-on: ubuntu-latest
17     permissions:
18       contents: read
19       packages: write
```

This screenshot shows the Visual Studio Code interface with the file explorer on the left. The main editor window is open to the file 'ci-cd.yml' in the path 'escenario-4-cicd > ejercicio > .github > workflows'. The code defines a GitHub Actions workflow named 'CI/CD con Tests y Seguridad'.

```
1 name: CI/CD con Tests y Seguridad
2
3 on:
4   push:
5     branches: [ main, develop ]
6     tags: [ "v*" ]
7   pull_request:
8     branches: [ main ]
9
10 env:
11   IMAGE_NAME: usuario-dockerhub/app-cicd-ejercicio
12
13 jobs:
14   test:
15     runs-on: ubuntu-latest
16     steps:
17       - name: Checkout del codigo
18         uses: actions/checkout@v4
```

This screenshot shows the Visual Studio Code interface with the file explorer on the left. The main editor window is open to the file 'test_app.py' in the path 'escenario-4-cicd > ejercicio > tests'. The code is a Python script using FastAPI's TestClient to test an application.

```
1 from fastapi.testclient import TestClient
2 import sys
3 import os
4
5 sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "src"))
6 from main import app
7
8 client = TestClient(app)
9
10 def test_index():
11     response = client.get("/")
12     assert response.status_code == 200
13     assert "mensaje" in response.json()
14
15 def test_health():
16     response = client.get("/health")
17     assert response.status_code == 200
18     assert response.json()["status"] == "healthy"
```

At the bottom, the terminal window shows the command 'docker ps' and its output, which includes 'powershell'.

Probando localmente el build multi-stage

Como el pipeline completo de GitHub Actions requiere configurar credenciales (`DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`) como *secrets* del repositorio —paso de configuración en la plataforma de GitHub, no en el código—, se probó localmente la parte que sí puede verificarse sin esas credenciales: el build multi-stage y el funcionamiento de la aplicación.

```
>> API en FastAPI con pipeline de CI/CD usando GitHub Actions. ...
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> cd escenario-4-cicd\ejercicio
>> docker build -t app-cicd-ejercicio .
>> docker run -d -p 8000:8000 --name test-cicd app-cicd-ejercicio
>> curl.exe http://localhost:8000/
⦿ >> curl.exe http://localhost:8000/health
[+] Building 48.2s (12/12) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.2s
=> => transferring dockerfile: 390B                               0.1s
=> [internal] load metadata for docker.io/library/python:3.11-slim 3.2s
=> [auth] library/python:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                   0.1s

⦿ >> curl.exe http://localhost:8000/health
{"mensaje":"Hola desde CI/CD con tests y multi-stage","version":"1.0.0"}{"status":"healthy"}
⦿ PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-4-cicd\ejercicio>
```

Esto confirma que la imagen construida con multi-stage build (etapa *build* + etapa *production*) funciona correctamente en tiempo de ejecución, con una imagen final más liviana al no incluir las herramientas de compilación de la primera etapa.

Configuración pendiente en GitHub (para producción real)

Para que el pipeline complete el push automático a DockerHub y GHCR, es necesario configurar en el repositorio (*Settings* → *Secrets and variables* → *Actions*) los secrets `DOCKERHUB_USERNAME` y `DOCKERHUB_TOKEN`. El secret `GITHUB_TOKEN` para GHCR se genera automáticamente por GitHub Actions y no requiere configuración manual.

Conclusión — Escenario 4

Este escenario introdujo la automatización del ciclo de construcción y publicación de imágenes mediante GitHub Actions. Se aplicaron dos buenas prácticas clave: correr las pruebas automatizadas antes de construir la imagen (*needs: test*), evitando publicar código que no pasa sus propias pruebas; y usar *multi-stage builds* para mantener las imágenes de producción livianas, separando las herramientas de compilación del entorno final de ejecución. También se incorporó seguridad al pipeline con el escaneo de vulnerabilidades vía Trivy, y redundancia de disponibilidad al publicar en dos registries distintos (DockerHub y GHCR).

Escenario 4 — Cierre: merge a main

Se hizo commit (`feat: agrega pipeline CI/CD con GitHub Actions`), push a la rama `escenario-4-cicd`, y merge fast-forward a `main` (12 archivos, 264 inserciones), sin conflictos

```
>> git add escenario-4-cicd/
>> git status
eplaced by CRLF the next time Git touches it
warning: in the working copy of 'escenario-4-cicd/ejemplo/src/package.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'escenario-4-cicd/ejemplo/src/server.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'escenario-4-cicd/ejercicio/.github/workflows/ci-cd.yml', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'escenario-4-cicd/ejercicio/Dockerfile', LF will be replaced by CRLF the next time Git touches it
```

```
>> git status
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git commit -m "feat: agrega pipeline CI/CD con GitHub Actions (Escenario 4)"
● >> git push origin escenario-4-cicd
>> git checkout main
>> git merge escenario-4-cicd
>> git push origin main
[escenario-4-cicd 75db944] feat: agrega pipeline CI/CD con GitHub Actions (Escenario 4)
12 files changed, 264 insertions(+)
create mode 100644 escenario-4-cicd/ejemplo/.github/workflows/docker-publish.yml
create mode 100644 escenario-4-cicd/ejemplo/Dockerfile
create mode 100644 escenario-4-cicd/ejemplo/src/healthcheck.js
```

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git commit -m "feat: agrega pipeline CI/CD con GitHub Actions (Escenario 4)"
>> git push origin escenario-4-cicd
>> git checkout main ...
create mode 100644 escenario-4-cicd/ejercicio/docker-compose.prod.yml
create mode 100644 escenario-4-cicd/ejercicio/src/main.py
create mode 100644 escenario-4-cicd/ejercicio/src/requirements.txt
create mode 100644 escenario-4-cicd/ejercicio/tests/test_app.py
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/robledocordoba123-cmyk/taller-docker-git.git
f7f8b32..75db944 main -> main
○ PS C:\Users\roble\OneDrive\Desktop\taller-docker-git>
```

Escenario 4 — Obstáculo: los workflows no se ejecutaban

Tras el primer push a `main`, la pestaña de GitHub Actions no mostraba ninguna ejecución. Se verificó primero que los permisos de Actions estuvieran habilitados (`Settings → Actions → General`, en "Allow all actions and reusable workflows" — correcto), descartando un problema de permisos.

La causa real: **GitHub Actions solo detecta workflows ubicados en `.github/workflows/` en la raíz del repositorio**, no en subcarpetas anidadas. Como los archivos estaban en `escenario-4-cicd/ejemplo/.github/workflows/` y

`escenario-4-cicd/ejercicio/.github/workflows/`, GitHub nunca los reconoció como workflows activos, sin importar cuántos commits o pushes se hicieran.

Solución: se movieron ambos archivos a `.github/workflows/` en la raíz (`docker-publish-ejemplo.yml` y `ci-cd-ejercicio.yml`), ajustando las rutas internas de `context:` en el paso de build (de `./ejemplo` a `./escenario-4-cicd/ejemplo`, por ejemplo) para que apunten correctamente a la nueva ubicación relativa. Se eliminaron las carpetas `.github` duplicadas dentro de cada escenario, y se hizo commit con `git add . && git commit -m "fix: mueve workflows a la raíz del repositorio (.github/workflows)"`.

Tras el push, ambos workflows aparecieron corriendo en la pestaña Actions, confirmando que la causa era efectivamente la ubicación de los archivos.

```
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> mkdir .github\workflows

Directorio: C:\Users\roble\OneDrive\Desktop\taller-docker-git\.github

Mode                LastWriteTime         Length Name
----                -
d-----          21/08/2026  12:38 a. m.             workflows

PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> @'
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> dir .github\workflows

Directorio: C:\Users\roble\OneDrive\Desktop\taller-docker-git\.github\workflows

Mode                LastWriteTime         Length Name
----                -
-a---l            21/08/2026  12:39 a. m.       2460 ci-cd-ejercicio.yml
-a---l            21/08/2026  12:39 a. m.       1419 docker-publish-ejemplo.yml
```

Escenario 4 — Obstáculo: incompatibilidad de versiones en los tests

El job `test` falló con `TypeError: Client.__init__() got an unexpected keyword argument 'app'`. La causa es que `TestClient` de FastAPI depende internamente de `httpx`, y las versiones de `httpx` 0.28 o superiores eliminaron el parámetro `app` que `TestClient` requiere. Como el workflow instalaba `httpx` sin fijar versión, tomaba automáticamente la última disponible (incompatible). Se solucionó fijando la versión con `pip install pytest "httpx<0.28"` en el paso de instalación de dependencias.



Help us improve GitHub Actions

Tell us how to make GitHub Actions work better for you with three quick questions.

Give feedback

2 workflow runs

Workflow ▾Event ▾Status ▾Branch ▾Actor ▾

✖

fix: mueve workflows a la raiz del repositorio (.github/workflows/)

CI/CD con Tests y Seguridad #1: Commit [ab1d440](#) pushed by [robledocordoba123-cmyk](#)

main

8 minutes ago

16s

...

✖

fix: mueve workflows a la raiz del repositorio (.github/workflows/)

Build y Push a DockerHub (Ejemplo) #1: Commit [ab1d440](#) pushed by [robledocordoba123-cmyk](#)

main

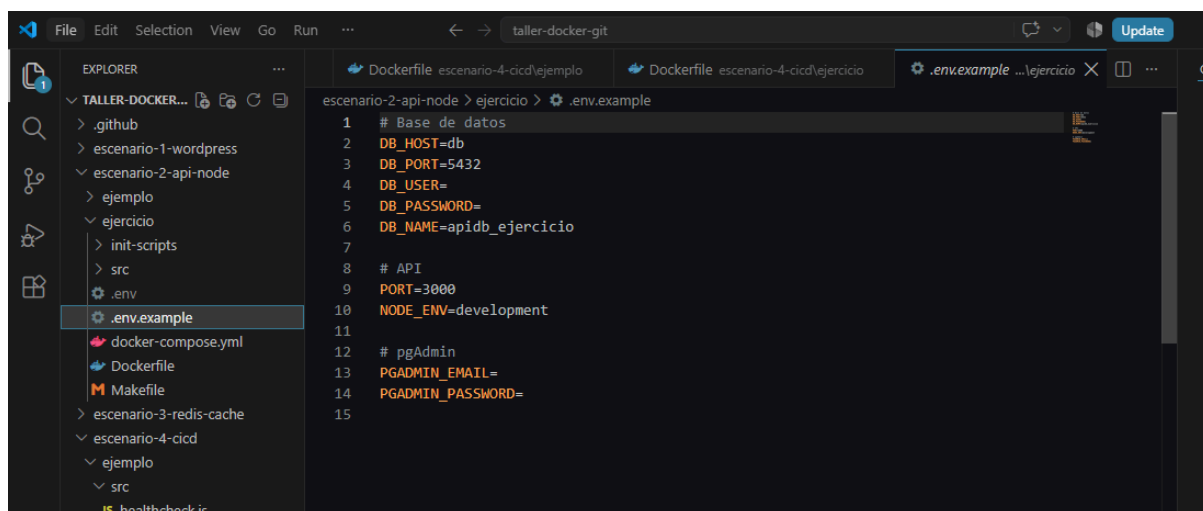
8 minutes ago

31s

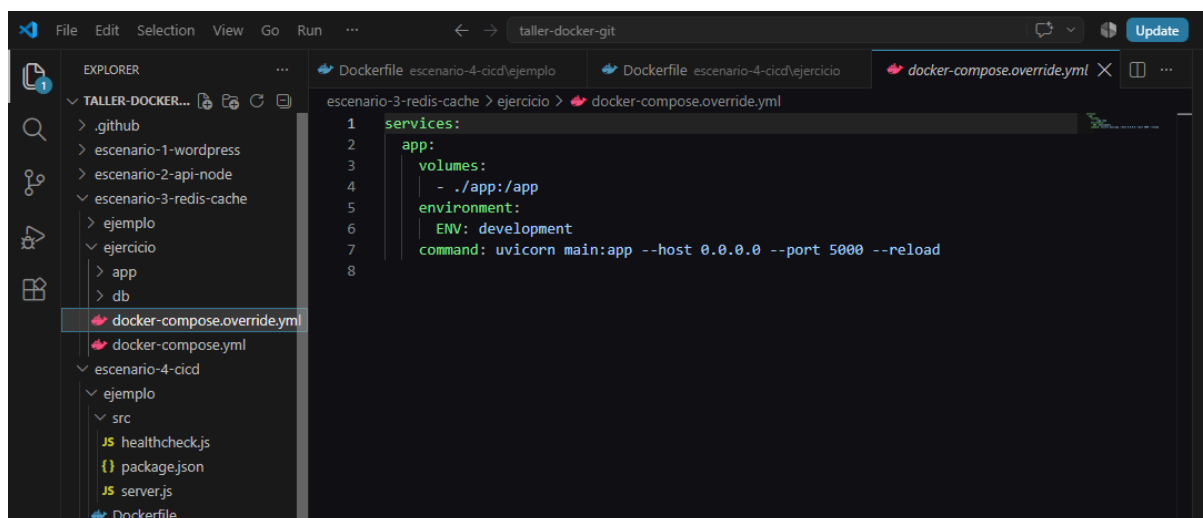
...

Tras corregir la ubicación de los workflows, la versión de `httpx`, el comando `npm ci` sin lockfile, y el typo en la ruta `.local` del multi-stage build, ambos workflows (**CI/CD con Tests y Seguridad** y **Build y Push a DockerHub**) se ejecutaron completos y en verde, confirmando que el pipeline corre las pruebas automatizadas, construye las imágenes con builds multi-etapa, las publica en DockerHub, y ejecuta el escaneo de seguridad con Trivy, tal como lo pedía la guía

Escenario 2 — Ajuste final: Se agregó `.env.example` como plantilla sin credenciales reales, siguiendo la misma buena práctica aplicada en el Escenario 1.



Escenario 3 — Ajuste final: Se agregó `docker-compose.override.yml`, que Docker Compose combina automáticamente con el `docker-compose.yml` base. Permite trabajar en modo desarrollo (código montado como volumen, recarga automática con `--reload`) sin modificar la configuración base pensada para producción.



Cierre general — verificación final: Se confirmó que los cuatro escenarios están correctamente integrados a `main`, que las carpetas generadas automáticamente (`node_modules`, `__pycache__`) nunca se subieron al repositorio gracias al `.gitignore`, y se completaron los últimos detalles pendientes de cada guía (`.env.example` en el Escenario 2, `docker-compose.override.yml` en el Escenario 3).

```

PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> Test-Path escenario-2-api-node\ejercicio\env.example
True
>> Test-Path escenario-3-redis-cache\ejercicio\docker-compose.override.yml
True
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git> git add .
>> git commit -m "docs: agrega .env.example en escenario 2 y docker-compose.override en escenario 3"
● >> git push origin main
warning: in the working copy of 'escenario-2-api-node/ejercicio/.env.example', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'escenario-3-redis-cache/ejercicio/docker-compose.override.yml', LF will be replaced by CRLF the next time Git touches it

```

Escenario 3 — Ajuste final: healthcheck para el servicio **app**

Se detectó que el servicio **app** no tenía healthcheck, incumpliendo el requisito de "healthchecks para todos los servicios". Al agregarlo apuntando al puerto 8000 (el definido en el **Dockerfile**), el contenedor quedó en estado **unhealthy**. Con **docker inspect --format "{{json .State.Health}}"** se identificó el error real: **Connection refused**. Revisando los logs (**docker logs**) se confirmó que **uvicorn** corría en el puerto **5000**, no 8000 — porque el **docker-compose.override.yml** (usado para desarrollo) sobrescribe el **command** original con **--port 5000 --reload**. Se corrigió el healthcheck para apuntar al puerto correcto, y tras reconstruir los contenedores, **docker-compose ps** confirmó los 3 servicios (**db**, **redis**, **app**) en estado **healthy**.

```

PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-3-redis-cache\ejercicio> docker-compose ps
NAME                IMAGE             COMMAND                  SERVICE    CREATED         STATUS
ejercicio3_app      ejercicio-app     "uvicorn main:app --..." app        About a minute ago Up About a minute (healthy)
ejercicio3_postgres postgres:15-alpine "docker-entrypoint.s..." db         About a minute ago Up About a minute (healthy)
ejercicio3_redis    redis:7-alpine    "docker-entrypoint.s..." redis     About a minute ago Up About a minute (healthy)
PS C:\Users\roble\OneDrive\Desktop\taller-docker-git\escenario-3-redis-cache\ejercicio> cd C:\Users\roble\OneDrive\Desktop\taller-docker-git
>> git add escenario-3-redis-cache\ejercicio\docker-compose.yml escenario-4-cicd\ejercicio\README.md

```

Escenario 4 — Ajuste final: Se agregó al **README.md** el badge de estado del workflow (**![CI/CD](...)/badge.svg**), cumpliendo el requisito 30 de la guía.

Files

main

Go to file

> .github

> escenario-1-wordpress

> escenario-2-api-node

> escenario-3-redis-cache

> escenario-4-cicd

> ejemplo

> **ejercicio**

> src

> tests

Dockerfile

README.md

docker-compose.prod.yml

.gitignore

README.md

taller-docker-git / escenario-4-cicd / ejercicio /

↑ Top

README.md

App CI/CD - Ejercicio

API en FastAPI con pipeline de CI/CD usando GitHub Actions.

Pipeline

- Ejecuta tests con pytest
- Si pasan, construye la imagen Docker (multi-stage build)
- Publica en DockerHub y GitHub Container Registry (GHCR)
- Escanea vulnerabilidades con Trivy

Correr localmente

`docker build -t app-cicd-ejercicio .` `docker run -p 8000:8000 app-cicd-ejercicio` `curl http://localhost:8000/health`

CI/CD con Tests y Seguridad passing

